



单位代码 10006

学 号 22373311

分 类 号 TP311

北京航空航天大学
BEIHANG UNIVERSITY

毕业设计 (论文)

基于 EDA 工具反馈的 LLM Verilog
生成与自动优化

学 院 名 称 计算机学院

专 业 名 称 计算机科学与技术

学 生 姓 名 张金涛

指 导 教 师 王锐

2026 年 5 月

北京航空航天大学

本科生毕业设计（论文）任务书

I、毕业设计（论文）题目：

基于 EDA 工具反馈的 LLM Verilog 生成与自动优化

II、毕业设计（论文）使用的原始资料（数据）及设计技术要求：

1. AutoChip 原论文

2. VerilogEval、RTLLM 评估基准

3. Icarus Verilog 开源 EDA 工具

4. 大语言模型 API（Claude、GPT 系列）

5. 北航本科毕业设计相关规范

III、毕业设计（论文）工作内容：

1. 复现 AutoChip 风格 RTL 代码生成与反馈修复系统

2. 接入 VerilogEval 和 RTLLM 双 benchmark

3. 在三个不同能力等级的 LLM 上验证反馈循环有效性

4. 通过消融实验分离反馈信息与多采样的贡献

5. 探索反馈粒度、多轮对话和提示词策略的影响

6. 建立实验产物管理与审计机制

IV、主要参考资料：

- [1] Thakur S, et al. AutoChip: Automating HDL Generation, 2023
- [2] Liu M, et al. VerilogEval: Evaluating LLMs for Verilog, ICCAD 2023
- [3] Lu Y, et al. RTLLM: Open-Source Benchmark for RTL Generation, 2024
- [4] Liu S, et al. RTLCoder: LLM-Assisted RTL Code Generation, 2024
- [5] Tsai Y, et al. RTLFixer: Fixing RTL Syntax Errors with LLMs, 2024
- [6] Yao Z, et al. HDLDebugger: Streamlining HDL Debugging, 2025
- [7] Wei J, et al. Chain-of-Thought Prompting, NeurIPS 2022
- [8] Brown T, et al. Language Models are Few-Shot Learners, NeurIPS 2020

____ 计算机 ____ 学院 ____ 计算机科学与技术 ____ 专业类 ____ 220611 ____ 班

学生 ____ 张金涛 ____

毕业设计(论文)时间： ____ 2025 ____ 年 ____ 12 ____ 月 ____ 29 ____ 日至 ____ 2026 ____ 年 ____ 5 ____ 月 ____ 22 ____ 日

答辩时间： ____ 2026 ____ 年 ____ 6 ____ 月 ____ 3 ____ 日

成 绩： _____

指导教师： _____

兼职教师或答疑教师（并指出所负责部分）：

____ 系（教研室）主任（签字）： _____

注：任务书应该附在已完成的毕业设计（论文）的首页。



本人声明

我声明，本论文及其研究工作是由本人在导师指导下独立完成的，在完成论文时所利用的一切资料均已在参考文献中列出。

作者： 张金涛

签字：

时间： 2026 年 5 月



基于 EDA 工具反馈的 LLM Verilog 生成与自动优化

学 生：张金涛

指导教师：王锐

摘 要

集成电路设计规模的持续扩大使得寄存器传输级（RTL）代码的开发与验证成为芯片设计流程中的关键瓶颈。大语言模型已在多种编程语言的代码生成任务中展现出潜力，但 RTL 代码的生成面临独特挑战——Verilog 代码描述的是并行执行的硬件结构，其正确性不仅取决于语法合规性，还必须经由 EDA 工具的编译和功能仿真验证。实际测试表明，当前模型在零样本条件下对复杂 Verilog 设计的正确率仍有较大提升空间。

本文构建了一个基于 EDA 工具反馈的 LLM Verilog 生成与自动优化系统。该系统实现了完整的“生成—编译—仿真—反馈—修复”迭代闭环：将 Icarus Verilog 编译器和 VVP 仿真器的输出解析为结构化反馈信号，引导大语言模型定向修复代码错误。系统支持 VerilogEval-Human 与 RTLLM 双 benchmark 接入，可通过统一 API 网关切换不同模型，并提供五级反馈粒度、多轮对话反馈和四种提示词策略的灵活配置。

实验方面，本文在三个不同能力等级的模型上执行了七组递进式对照实验。在 VerilogEval-Human 20 题子集上，Haiku 的通过率从 80% 提升至 90%；在难度更高的 RTLLM_STUDY_12（12 道精选设计）上，GPT-5.4 从 50% 提升至 83%。消融实验定量分离了反馈收益的来源——错误反馈信息贡献约 57% 的提升，多采样贡献约 43%，且存在仅反馈可解而 15 次独立重试均无法通过的设计题目。

稳定性实验（4 轮重复）证实了核心结论的可复现性。反馈粒度实验揭示了信息量与修复效率之间的倒 U 型关系：编译反馈和简洁反馈最为稳健，过于详尽的反馈反而可能干扰模型的修复判断。实验还发现反馈效果受模型能力制约，不同模型对反馈信息的利用效率存在显著差异。

关键词：大语言模型；Verilog 代码生成；EDA 工具反馈；自动优化；RTLLM



LLM Verilog Generation and Automatic Optimization Based on EDA Tool Feedback

Author: Zhang Jintao

Tutor: Wang Rui

Abstract

The growing complexity of integrated circuit design makes Register Transfer Level (RTL) code development and verification a major bottleneck in the chip design workflow. While Large Language Models (LLMs) have demonstrated code-generation capabilities across several programming languages, generating hardware description code poses unique challenges: Verilog describes parallel hardware structures whose correctness must be verified through Electronic Design Automation (EDA) tool compilation and functional simulation, rather than simple execution.

This thesis presents a prototype system for EDA-feedback-driven automatic RTL generation and optimization. The system parses Icarus Verilog compiler and VVP simulator outputs into structured feedback signals that guide the LLM toward targeted code fixes. It supports two public benchmarks (VerilogEval-Human and RTLLM), multi-model switching via a unified API gateway, five levels of feedback granularity, multi-turn conversational feedback, and four prompt engineering strategies.

Seven progressive experiments were conducted on three models of different capability tiers. On VerilogEval-Human (20-problem subset), Haiku's pass rate improved from 80% to 90%. On the more demanding RTLLM_STUDY_12 (12 curated designs), GPT-5.4 rose from 50% to 83%. Ablation experiments quantitatively decomposed the feedback benefit: error feedback contributed roughly 57% of the improvement, while multi-sampling accounted for the remaining 43%. Notably, certain designs could only be solved with feedback and resisted 15 independent retry attempts.

Repeated trials (four runs per model) confirmed the reproducibility of the core findings. A feedback-granularity sweep revealed an inverted-U relationship between information volume and repair efficiency: compile-only and succinct feedback were the most robust choices,



whereas excessively detailed feedback risked overwhelming the model's repair reasoning. The experiments further showed that feedback effectiveness is modulated by model capability, with different models exhibiting markedly different feedback utilization patterns.

Key words: Large Language Models; Verilog Generation; EDA Tool Feedback; Automatic Optimization; RTLLM



目 录

1 绪论	5
1.1 研究背景与意义	5
1.2 国内外研究现状	5
1.2.1 LLM 辅助 RTL 代码生成	5
1.2.2 EDA 工具反馈与自动修复	6
1.2.3 现有工作的不足	6
1.3 本文研究内容	6
1.4 本文主要贡献	7
1.5 论文组织结构	7
2 相关技术与研究基础	8
2.1 Verilog 与 RTL 设计基础	8
2.1.1 RTL 设计的基本概念	8
2.1.2 RTL 代码与软件代码的关键差异	8
2.2 大语言模型代码生成基础	9
2.2.1 Transformer 架构与预训练	9
2.2.2 指令跟随与对齐	9
2.2.3 LLM 生成 RTL 代码的常见问题	9
2.3 EDA 编译与仿真验证流程	10
2.3.1 编译验证	10
2.3.2 功能仿真	10
2.3.3 编译仿真反馈的信号价值	10
2.4 AutoChip 方法与 EDA Feedback Loop	11
2.4.1 AutoChip 的核心思想	11
2.4.2 反馈信息的组织	11
2.4.3 多候选生成与全局最优追踪	11



2.4.4 本文与 AutoChip 的关系	11
2.5 RTL 生成评估基准	12
2.5.1 VerilogEval Benchmark	12
2.5.2 RTLLM Benchmark	12
2.6 相关研究与本文定位	12
2.6.1 相关研究方向	12
2.6.2 本文定位	13
2.7 本章小结	13
3 系统设计与实现	14
3.1 系统总体架构	14
3.1.1 设计目标与约束	14
3.1.2 模块划分与数据流	14
3.2 任务加载与 Benchmark 接入	16
3.2.1 统一 Task 数据结构	16
3.2.2 RTLLM 加载器的目录发现逻辑	16
3.2.3 VerilogEval 加载器	16
3.3 大语言模型调用与模型管理	16
3.3.1 统一 API 网关架构	16
3.3.2 自动重试与错误分类	17
3.4 Verilog 代码提取、编译与仿真执行	17
3.4.1 代码提取策略	17
3.4.2 编译执行	17
3.4.3 仿真执行与双格式解析	17
3.4.4 评分机制	18
3.5 反馈循环控制机制	18
3.5.1 全局最优追踪	18
3.5.2 Zero-shot 与 Retry-only 作为退化模式	18
3.5.3 API 错误容错	21



3.6 反馈策略扩展实现	21
3.6.1 五级反馈粒度	21
3.6.2 多轮对话反馈	21
3.7 提示词策略扩展实现	22
3.7.1 四种提示策略	22
3.7.2 数据泄漏防护	22
3.7.3 策略调度与元数据记录	22
3.8 实验产物管理与审计机制	22
3.8.1 运行目录与产物文件	22
3.8.2 元数据与 Git 集成	23
3.8.3 审计脚本	23
3.9 本章小结	23
4 实验设计	24
4.1 实验目标与总体设计	24
4.2 评估基准与任务子集	25
4.3 模型与参数设置	25
4.4 实验条件与对照设计	27
4.5 评价指标与统计方法	27
4.6 实验可靠性控制	27
4.7 本章小结	28
5 实验结果与分析	29
5.1 VerilogEval-Human 基线实验	29
5.2 RTLLM_STUDY_12 正式实验结果	30
5.3 Feedback vs Retry-only 消融实验	31
5.4 稳定性重复实验	33
5.5 反馈粒度实验	34
5.6 多轮对话反馈实验	36
5.7 提示词策略实验	36



5.8 典型案例分析	40
5.9 综合讨论	40
5.10 本章小结	42
6 总结与展望	43
6.1 本文工作总结	43
6.2 主要实验结论	43
6.3 系统局限性	44
6.4 后续工作展望	44
6.5 本章小结	45
结论	46
致谢	47
参考文献	48
附录 A 实验配置与运行命令	50
A.1 正式实验运行命令	50
A.2 环境配置	51
A.3 核心代码模块索引	51



1 绪论

1.1 研究背景与意义

随着集成电路设计规模的持续增长，寄存器传输级（RTL）代码的编写和验证已成为芯片设计流程中最耗时的环节之一。传统的 RTL 开发高度依赖工程师的专业经验——设计者需要根据功能规格手工编写 Verilog 或 VHDL 代码，并通过反复的编译、仿真和调试迭代来确保功能正确性。这一过程周期长、人力成本高，且容易因疏忽引入难以发现的逻辑错误。

近年来，大语言模型（Large Language Model, LLM）在软件代码生成领域展现出一定能力，能够根据自然语言描述生成多种编程语言的代码。这一进展自然引出了一个研究问题：大语言模型能否有效地生成硬件描述语言代码，从而辅助甚至部分替代人工的 RTL 开发过程？

然而，RTL 代码生成与普通软件代码生成存在本质差异。RTL 代码描述的是并行执行的硬件结构而非顺序执行的软件逻辑，正确的 RTL 设计不仅要求语法正确，还必须满足时序约束、位宽匹配、时钟域一致性等硬件特有要求，并通过 EDA 工具的编译和仿真验证。在零样本条件下，大语言模型生成的 RTL 代码往往存在编译错误、接口不匹配、边界条件遗漏等问题，尤其在涉及复杂状态机或浮点运算等场景时正确率较低。

这一现实使得将大语言模型的生成能力与 EDA 工具的验证能力相结合成为一个值得探索的方向。通过构建“生成—编译—仿真—反馈—修复”的迭代闭环，EDA 工具的编译和仿真输出可以作为结构化的反馈信号引导模型定向修复代码错误，有望提升 RTL 代码的自动生成质量。

1.2 国内外研究现状

1.2.1 LLM 辅助 RTL 代码生成

已有研究探索了使用大语言模型从自然语言规格直接生成 Verilog 代码的可行性 [1-2]。在评估基准方面，NVlabs 发布的 VerilogEval[3] 提供了标准化的 spec-to-RTL 评估框架，香港科技大学发布的 RTLLM[4] 则提供了更高难度的设计任务集。这些基准的建立推动了 RTL 生成方法的标准化评估和横向比较。



现有研究表明,大语言模型在简单的组合逻辑和常见时序电路上具备一定的生成能力,但在涉及复杂算法、多状态 FSM 或精确位操作的设计上,单次生成的正确率仍有较大提升空间。

1.2.2 EDA 工具反馈与自动修复

AutoChip[5]等工作提出了利用 EDA 工具反馈驱动代码修复的方法,将传统的单次生成扩展为迭代修复过程。该类方法的核心思想是将编译器和仿真器输出的错误信息提取为结构化反馈,使模型能够进行定向修复而非盲目重新生成。RTLFixer[6]针对编译错误修复进行了专门探索,HDLError[7]引入了检索增强技术辅助调试,这些研究共同体现了“验证反馈驱动修复”的研究趋势。

1.2.3 现有工作的不足

尽管上述研究在方法探索和概念验证方面取得了有价值的进展,仍有几个方面值得进一步分析。

在反馈收益的来源方面,反馈循环带来的通过率提升中有多少来自错误信息的引导、有多少仅来自多次重试的采样收益,现有工作较少对此进行严格的消融分析。在强模型与高难度场景方面,部分研究基于较早的模型或较简单的 benchmark 进行评估,需要新的实验证据。在反馈机制的影响因素方面,反馈粒度、反馈组织方式和初始提示词策略等维度对修复效果的影响,尚未得到系统的实验研究。此外,部分工作在实验结果管理和结论稳定性验证方面的机制不够完善。

1.3 本文研究内容

针对上述不足,本文的工作包含系统构建和实验研究两部分。

在系统构建方面,本文实现了一个基于 EDA 工具反馈的 LLM Verilog 生成与自动优化原型系统。该系统以 Python 编写,以 Icarus Verilog 作为 EDA 后端,实现了完整的“生成—编译—仿真—反馈—修复”闭环,并支持 VerilogEval 和 RTLLM 双 benchmark 适配、统一 API 网关下的多模型调用、五级反馈粒度和四种提示词策略的灵活组合。此外,对 RTLLM 2.0 全部 50 个设计进行了 Icarus Verilog 兼容性审计(41/50 完全可用),从中精选 12 道高难度设计构成 RTLLM_STUDY_12 正式研究子集。

在实验研究方面,使用 Haiku、Sonnet 4.6 和 GPT-5.4 三个不同能力等级的模型,执



行了七组递进式对照实验。实验从基础有效性验证起步,经由 `retry-only` 消融实验分离多采样与反馈信息各自的贡献,进而通过 4 轮重复实验检验结论的统计稳定性。在此基础上,分别探索了反馈粒度、多轮对话模式和提示词策略对修复效果的影响。全部实验均配有自动化产物管理和审计机制。

1.4 本文主要贡献

本文的贡献可从三个角度概括。

在工程实现方面,完成了一个可复现的基于 EDA 工具反馈的 RTL 生成与优化原型系统,支持 VerilogEval 与 RTLLM 双 benchmark、统一 API 网关下的多模型调用、五级反馈粒度与四种提示词策略的组合配置。系统内置了实验产物自动序列化和数据审计脚本,确保每次实验可追溯至具体代码版本。围绕核心系统,项目还实现了 RTLLM 兼容性扫描、多模式实验运行和结果图表整理等配套脚本,使后续多组对照实验能够自动化执行和复现。

在实验验证方面,以 RTLLM_STUDY_12 作为核心任务集,在更高难度场景下验证了反馈循环的有效性。通过 `retry-only` 消融实验将反馈收益定量分解为多采样和错误反馈信息两部分,并通过 4 轮独立重复实验确认了结论的可复现性。实验中还观察到反馈效果的模型依赖现象:GPT-5.4 上反馈始终带来正向增益,而 Sonnet 4.6 上反馈在当前配置下未能超越纯重试。

在机制探索方面,通过五级粒度实验揭示了反馈信息量与修复效率之间的倒 U 型关系——编译反馈和简洁反馈的稳健性最强,过于详尽的反馈可能引起信息过载而降低修复成功率。

1.5 论文组织结构

本文共分六章。第 1 章为绪论,介绍研究背景、现状和本文工作。第 2 章为相关技术与研究基础,介绍 Verilog 与 RTL 设计基础、大语言模型代码生成机制、EDA 编译仿真流程和反馈循环方法。第 3 章为系统设计与实现,描述原型系统的架构和关键模块。第 4 章为实验设计,定义全部实验方案和评价指标。第 5 章为实验结果与分析,呈现七组实验的结果并进行综合讨论。第 6 章为总结与展望,总结工作、讨论局限性并展望后续方向。



2 相关技术与研究基础

本章介绍与本文研究密切相关的技术基础和评估基准。内容涵盖 Verilog 硬件描述语言的设计要素、大语言模型的代码生成机制、EDA 工具链的编译仿真流程、AutoChip 方法论,以及本文实验所依赖的两个公开评估基准。上述内容为后续第 3 章的系统设计和第 4-5 章的实验分析提供技术背景。

2.1 Verilog 与 RTL 设计基础

2.1.1 RTL 设计的基本概念

寄存器传输级(Register Transfer Level, RTL)是数字集成电路设计中广泛采用的抽象层次[8]。在 RTL 层面,设计者以硬件描述语言刻画数据在寄存器之间的传输路径和变换逻辑,综合工具随后将其转化为门级网表并最终映射到物理版图。IEEE 1364-2005 标准[8]定义了 Verilog 语言的语法和语义规范,为 RTL 设计提供了工业级的标准化基础。

Verilog 的基本设计单元是模块(module)。每个模块声明一组输入/输出端口,并在模块体内实现硬件功能。按照信号驱动方式的不同,模块内部逻辑可分为两类:组合逻辑(其输出仅由当前输入决定,如加法器和多路选择器)与时序逻辑(其输出还受到过去状态的影响,如计数器和有限状态机)。有限状态机(FSM)是时序逻辑的典型代表,在协议解析、序列检测等数字系统控制场景中有广泛应用。

2.1.2 RTL 代码与软件代码的关键差异

RTL 代码的自动生成比普通软件代码更具挑战性,根本原因在于两者的执行模型截然不同。在并行性层面,C/Python 等软件语言的语句按程序顺序依次执行,而 Verilog 中的 always 块和连续赋值 assign 在仿真语义下并发执行,生成代码时必须正确处理多个信号之间的时序依赖关系。在时钟与复位层面,时序逻辑的行为由时钟边沿触发并依赖复位信号完成初始化,遗漏 posedge clk 敏感列表项或缺少复位分支是 LLM 生成 Verilog 时的高频错误。在位宽语义层面,硬件信号具有确定的位宽,符号扩展、截断和位拼接操作的疏漏会导致仿真结果与预期不一致。上述差异决定了 RTL 代码生成不能简单套用软件代码生成的范式,EDA 工具的编译和仿真验证环节不可或缺。



2.2 大语言模型代码生成基础

2.2.1 Transformer 架构与预训练

当代大语言模型的技术基础是 Transformer 架构 [9]。Vaswani 等人于 2017 年提出的自注意力机制 (Self-Attention) 使得模型能够在序列的任意位置之间建立直接关联, 解决了传统循环网络在长序列建模上的效率瓶颈。后续工作在此基础上进行了大规模扩展: GPT-3[10] 将参数量提升至 1750 亿, 通过在海量文本上的无监督预训练获得了上下文学习 (in-context learning) 能力。

在代码领域, OpenAI 的 Codex[11] 通过在 GitHub 公开代码库上对 GPT 进行微调, 显著提升了代码生成的通过率。Codex 同时提出了 HumanEval 基准和 pass@k 评估指标, 为代码生成研究建立了标准化的评估方法论。pass@k 指标衡量在 k 次独立采样中至少出现一个正确解的概率, 本文的多候选生成策略 ($k = 3$) 即借鉴了这一思路。Meta 的 Code Llama[12] 则证明了开源代码模型在 HumanEval 上也能达到接近闭源模型的水平。

2.2.2 指令跟随与对齐

经过预训练的基础模型并不天然擅长理解和遵循用户指令。Ouyang 等人 [13] 提出了基于人类反馈的强化学习 (RLHF) 方法, 通过收集人类偏好标注训练奖励模型, 再以此引导语言模型的生成行为使其更好地对齐用户意图。这一范式已成为当代闭源商业模型 (如 Claude、GPT-4 系列) 的核心训练流程之一。在本文的实验场景中, 模型需要理解自然语言形式的硬件功能描述并生成符合规格的 Verilog 代码, 指令跟随能力的优劣直接影响首轮生成的质量。

2.2.3 LLM 生成 RTL 代码的常见问题

尽管大语言模型具备一定的 Verilog 代码生成能力, 实际使用中仍频繁出现以下问题: 语法层面的错误 (模块声明不完整、端口列表与描述不匹配、信号未声明等) 导致 EDA 编译失败; 接口层面的不匹配 (生成的模块名称或端口定义与测试平台不一致) 导致模块实例化失败; 功能层面的边界条件错误 (代码在多数测试向量上正确但在特定边界条件上输出错误结果); 以及算法层面的理解偏差 (模型对复杂数学运算或状态转移逻辑的实现不够精确)。

上述问题在难度较高的设计任务上尤为突出, 这为引入 EDA 编译和仿真反馈来迭



代修复代码提供了直接动机。

2.3 EDA 编译与仿真验证流程

2.3.1 编译验证

硬件设计的编译阶段将 Verilog 源代码解析为仿真模型，并检查语法正确性、信号声明一致性和模块接口匹配性。编译器输出的错误信息能够精确定位代码中的结构性问题，例如未声明信号、端口数量不匹配或语法关键字拼写错误等。

本文选用 Icarus Verilog[14] 作为编译和仿真工具。Icarus Verilog 是一个遵循 IEEE 1364-2005[8] 标准的开源 Verilog 编译器与仿真器，通过 -g2012 选项还可启用部分 SystemVerilog 扩展特性。选择开源 EDA 工具的考虑包括：免费获取，适合本科毕设的硬件和许可条件；命令行接口简洁，易于嵌入自动化脚本；实验结果完全可复现。在工业实践中，Yosys[15] 等开源综合工具也已在学术界和小型 FPGA 项目中得到采用，形成了从编译仿真到逻辑综合的开源 EDA 工具链。

2.3.2 功能仿真

编译成功后，使用 VVP 仿真引擎执行编译产生的仿真二进制文件。仿真过程中，测试平台（testbench）对待测设计施加激励信号并将输出与预期结果逐一比对。

VerilogEval 和 RTLLM 的测试平台采用了不同的输出格式：VerilogEval 通过逐样本比较待测模块与参考模块的输出并报告 “mismatched samples is N out of M”；RTLLM 则直接输出 “Your Design Passed” 或失败统计 “Test completed with N/M failures”。这些结构化的仿真输出使反馈提取成为可能。

2.3.3 编译仿真反馈的信号价值

编译器和仿真器的输出天然携带有价值的调试线索：编译错误精确标识出语法或结构层面的缺陷位置和类型，仿真不匹配则揭示逻辑层面的功能偏差。将这些信息结构化后反馈给大语言模型，可以使模型的修复行为从盲目重写转变为定向修补。这一闭环思想构成了 AutoChip 方法的基础。



2.4 AutoChip 方法与 EDA Feedback Loop

2.4.1 AutoChip 的核心思想

AutoChip[5] 提出了一种基于 EDA 工具反馈的 Verilog 代码迭代修复方法。其核心流程为：大语言模型根据自然语言描述生成初始 Verilog 代码 → EDA 工具编译并仿真验证 → 提取错误信息构造结构化反馈 → 将反馈与任务描述合并为新提示词 → 模型生成修复版代码。此过程反复执行，直至代码通过全部测试或迭代次数耗尽。

与单次零样本生成相比，该闭环具有两方面优势：其一，利用 EDA 工具提供的精确错误定位信息，避免模型在已正确的部分做不必要的修改；其二，通过多轮迭代逐步逼近正确解，在模型首次生成“接近正确但存在局部错误”的情况下尤为有效。

2.4.2 反馈信息的组织

反馈信息的组织粒度直接影响模型的修复效率。一个极端是将完整的编译日志和全部仿真波形传递给模型，但这会消耗大量上下文窗口并可能引入噪声。另一个极端是仅告知“编译失败”或“仿真未通过”，但这缺少足够的修复线索。AutoChip 论文采用的简洁反馈 (succinct feedback) 策略介于两者之间：提取错误行号、不匹配样本数和仿真输出摘要，在控制输入长度的同时保留关键信息。

2.4.3 多候选生成与全局最优追踪

在每轮迭代中生成多个候选代码 (k-candidate)，并从中选出编译/仿真评分最高的候选作为当前最优。全局最优追踪机制跨轮次维护历史最佳候选，确保下一轮反馈基于历史最优的错误信息构建，防止因单轮质量波动而丢失已有的最优解。该策略与代码生成中的 pass@k 评估方法 [11] 具有理念相通之处。

2.4.4 本文与 AutoChip 的关系

本文复现了 AutoChip 的核心闭环思想，并在以下维度进行了扩展和深化：在评估范围上，同时接入 VerilogEval 和 RTLLM 两个公开 benchmark；在模型覆盖上，使用 Haiku、Sonnet 4.6 和 GPT-5.4 三个不同能力层级的模型进行交叉验证；在实验方法上，设计了 retry-only 消融条件以定量分离多采样收益与反馈信息收益。

此外，本文还通过 4 轮重复实验验证结论的统计稳健性，并在反馈粒度、多轮对话



和提示词策略等维度进行了系统探索。

2.5 RTL 生成评估基准

2.5.1 VerilogEval Benchmark

VerilogEval[3] 是 NVIDIA 研究团队发布的 Verilog 代码生成评估基准，发表于 IC-CAD 2023。其人工编写子集（VerilogEval-Human）从 HDLBits[16] 在线练习平台中选取了 156 道硬件设计任务，涵盖组合逻辑、时序逻辑、有限状态机等多种类型，难度从基础的线连接到 Conway 生命游戏等复杂设计。VerilogEval-Human v2[17] 进一步引入了规格到 RTL（specification-to-RTL）的评估范式。

VerilogEval 的评估方式为：给定自然语言描述和模块接口定义，要求模型生成完整的模块实现代码。本文在中期阶段选取 VerilogEval-Human 的 20 道题目作为基线实验任务集。

2.5.2 RTLLM Benchmark

RTLLM[4] 是香港科技大学发布的 RTL 设计生成评估基准，发表于 ASP-DAC 2024。该基准包含 50 个硬件设计任务，按功能类别划分为算术运算（Arithmetic）、控制逻辑（Control）、存储器（Memory）和杂项（Miscellaneous）四大类，难度从基础加法器到 IEEE 754 浮点乘法器、流水线乘法器和交通灯控制器等工程复杂度较高的设计。

相比 VerilogEval，RTLLM 的任务普遍在模块规模和算法复杂度上更具挑战性。本文在接入 RTLLM 前，对全部 50 个设计进行了 Icarus Verilog 兼容性审计：50 个设计中 46 个编译通过（92%），41 个仿真通过（82%）。在 41 个完全可用的设计中，精选了 12 道高难度、多类别覆盖的设计构成 RTLLM_STUDY_12 正式研究子集。

2.6 相关研究与本文定位

2.6.1 相关研究方向

围绕大语言模型与硬件设计的交叉领域，当前研究主要分布在以下几条技术路线上。

在 LLM 直接生成 RTL 方面，ChipGPT[1] 探索了从自然语言到 RTL 的端到端生成流程，Verigen[2] 在 DATE 2023 上展示了通过微调生成 Verilog 的可行性，RTLCoder[18]



则通过专门的 RTL 代码微调显著提升了开源模型的 RTL 生成质量。在评估基准方面, VerilogEval[3] 和 RTLLM[4] 的相继发布为该领域建立了标准化的评测框架。在 EDA 反馈驱动的代码修复方面, AutoChip[5] 提出了反馈闭环范式, RTLFixer[6] 针对编译错误修复做了专门研究, HDLDebugger[7] 引入了检索增强技术辅助硬件代码调试, AIvriI[19] 则探索了更精细的 verification-in-the-loop 策略。在多轮交互与代理方法方面, Verilog-Coder[20] 采用图规划和 AST 工具链实现了更复杂的代码修正流程, Chip-Chat[21] 讨论了对话式硬件设计的挑战与机遇。AutoBench[22] 则将 LLM 应用于测试平台的自动生成, 拓展了 EDA 自动化的边界。

2.6.2 本文定位

本文的定位是: 在 AutoChip 反馈闭环框架的基础上, 构建一个可复现的实验平台, 并在更强模型和更高难度的 benchmark 上, 系统地分析反馈循环的有效性及其影响因素。与上述相关工作相比, 本文的特色在于: 提供了严格的消融实验设计以分离反馈信息与多采样的各自贡献, 并通过重复实验验证结论的统计稳定性。

2.7 本章小结

本章从 Verilog 语言标准 [8] 和 RTL 设计基本概念出发, 介绍了 Transformer 架构 [9]、代码生成预训练 [11] 和指令对齐 [13] 等大语言模型技术基础, 梳理了基于 Icarus Verilog[14] 和 Yosys[15] 等开源工具的 EDA 编译仿真流程, 阐述了 AutoChip[5] 方法的核心闭环思想, 并分别介绍了 VerilogEval[3] 和 RTLLM[4] 两个评估基准的设计与选题策略。上述技术基础为下一章的系统设计与实现奠定了必要背景。



3 系统设计与实现

本章描述本文所构建的基于 EDA 工具反馈的 LLM Verilog 生成与自动优化原型系统。系统采用流水线式模块架构，以 Python 实现核心逻辑，以 Icarus Verilog[14] 作为 EDA 后端。其中，核心系统代码约 2100 行，覆盖任务加载、模型调用、Verilog 提取、编译仿真、评分和反馈循环等模块。除此之外，项目还实现了 RTLLM 兼容性扫描、正式实验运行与多种实验模式（消融、稳定性重复、反馈粒度、多轮对话、提示词策略）的调度脚本，用于支撑后续多模型、多条件的自动化实验。下文从总体架构出发，逐模块介绍设计决策和关键实现细节。

3.1 系统总体架构

3.1.1 设计目标与约束

系统的设计目标可概括为四个方面：（1）能够接入 VerilogEval[3] 和 RTLLM[4] 两种格式不同的公开 benchmark，并将题目统一为内部表示；（2）能够在同一 API 端点下切换不同的大语言模型，无需修改代码；（3）支持反馈粒度、提示词策略和对话模式的灵活配置，以覆盖多维度实验需求；（4）自动记录完整的实验元数据，使任何一次运行的结果都可追溯到具体的代码版本和参数设置。

在工程约束方面，系统需要运行在本科毕设可获取的硬件和软件环境下——这意味着不依赖商业 EDA 工具许可，不要求 GPU 本地部署，且全部依赖均可通过 pip 安装。

3.1.2 模块划分与数据流

系统由八个核心模块组成，其职责和对应的代码文件如表3.1所示。

数据在模块间的流转路径为：任务加载器 → 提示词构建器 → LLM 客户端 → Verilog 提取器 → Verilog 执行器 → 评分器 → 反馈循环控制器（判断是否继续迭代） → 产物管理器。图3.1展示了这一数据流及模块间的调用关系。



表 3.1 系统核心模块与代码文件映射

模块	职责	代码文件
任务加载器	从 benchmark 目录读取题目	rtllm_loader.py, verilogeval_loader.py
提示词构建器	组装初始/反馈提示词	prompt_builder.py
LLM 客户端	封装 API 调用与重试	client.py
Verilog 提取器	从模型回复中提取代码	extract_verilog.py
Verilog 执行器	调用 iverilog/vvp	verilog_executor.py
评分器	编译/仿真结果 → 标量分数	ranker.py
反馈循环控制器	迭代闭环主逻辑	loop_runner.py
产物管理器	元数据记录与序列化	artifacts.py

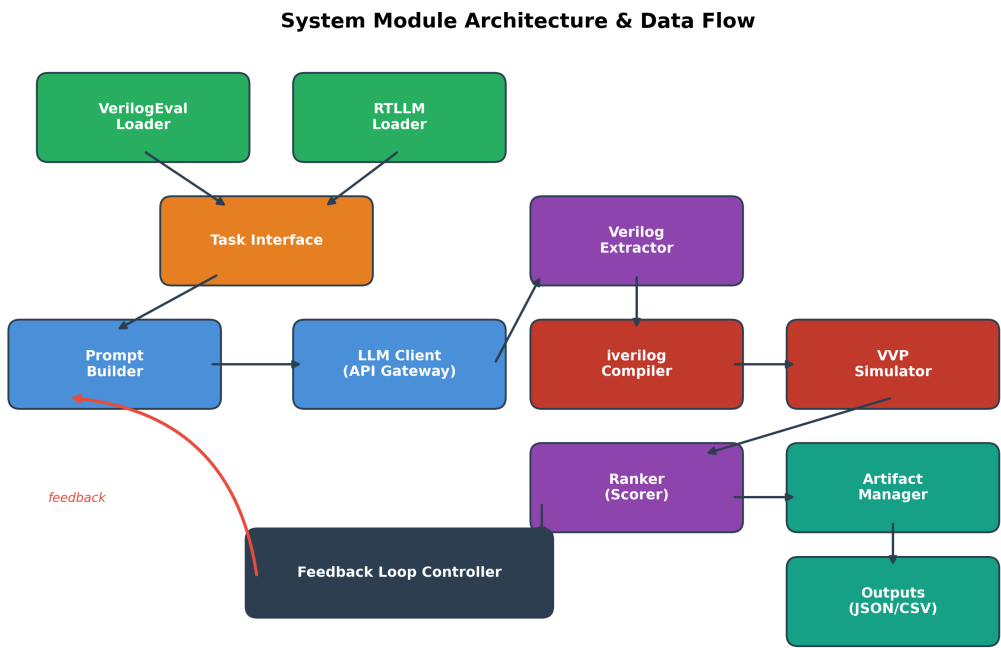


图 3.1 系统模块架构与数据流



3.2 任务加载与 Benchmark 接入

3.2.1 统一 Task 数据结构

为屏蔽不同 benchmark 在文件组织和题目格式上的差异，系统定义了统一的 Task 数据结构。该结构包含四个必需字段：name（任务名称，如 adder_16bit）、description（自然语言功能描述）、module_header（Verilog 模块的端口声明）、testbench_path（测试平台文件的绝对路径）。所有下游模块（提示词构建器、执行器、评分器等）仅依赖 Task 接口，不感知数据来源。

3.2.2 RTLLM 加载器的目录发现逻辑

RTLLM 2.0 的题目分布在四个顶层分类目录（Arithmetic、Control、Memory、Miscellaneous）下的嵌套子目录中。加载器通过递归遍历分类目录，对每个同时包含 design_description.txt 和 testbench.v 的目录创建一个 RTLLMProblem 对象。模块名称从设计描述文件中通过正则表达式 Module name:\s*\n?\s*(\w+) 提取，提取失败时回退为 top_module。此外，加载器还会搜索 verified_*.v 格式的参考实现文件，供分析使用。

3.2.3 VerilogEval 加载器

VerilogEval 采用 spec-to-RTL 模式，每道题目提供自然语言描述和 TopModule 接口定义。加载器读取描述文本作为 description，读取接口声明作为 module_header，并将参考实现与测试平台合并写入临时文件作为 testbench_path。两个加载器在输出端均产生统一的 Task 对象，使后续流程无需区分数据来源。

3.3 大语言模型调用与模型管理

3.3.1 统一 API 网关架构

系统通过第三方统一 API 网关接入多种大语言模型。网关的核心价值在于：API 密钥和服务端点保持不变，仅通过环境变量中的模型名称参数切换底层模型。模型名称的优先级链为：LLM_MODEL > ANTHROPIC_MODEL > 代码中的默认值。这一设计使得在同一实验脚本中切换 Haiku、Sonnet 和 GPT-5.4 只需修改一个环境变量。



3.3.2 自动重试与错误分类

API 调用在生产环境中不可避免地会遇到瞬态错误（网络超时、服务端过载、模型路由暂时不可用等）。系统实现了指数退避重试机制：对 HTTP 状态码 429、500–504、529 以及 “model_not_found” 等可重试错误，依次等待 2 秒、5 秒和 10 秒后重试，最多重试 3 次。当所有重试均失败时，错误被封装为 `APIError` 对象，记录错误类型和原始异常信息，写入该候选的结果记录中。这一设计确保了瞬态 API 故障不会导致整个实验批次崩溃，同时也不会将错误结果混入有效数据。

3.4 Verilog 代码提取、编译与仿真执行

3.4.1 代码提取策略

大语言模型的回复通常包裹在 Markdown 代码围栏（“`verilog ...`”）中，且在使用 CoT 提示策略时回复中还会穿插解释性文本。Verilog 提取器首先使用正则表达式移除所有 Markdown 围栏标记，然后通过 `module\s+\w+...endmodule` 模式匹配提取全部 Verilog 模块块。该策略对不同提示策略产生的回复格式均具有兼容性——无论模型是直接返回代码还是在代码前后添加解释，提取器都能正确定位 `module` 到 `endmodule` 之间的有效代码。

3.4.2 编译执行

编译阶段调用 Icarus Verilog，将生成的设计代码文件与测试平台文件一同编译。编译命令为 `iverilog -g2012 -o sim.out design.v testbench.v`，其中 `-g2012` 选项启用 SystemVerilog 2012 扩展特性以支持 RTLLM 中使用 `logic` 等新语法的测试平台。编译超时设为 30 秒。编译结果被封装为 `CompileResult` 对象，包含成功标志、标准输出和标准错误。

3.4.3 仿真执行与双格式解析

编译成功后，使用 VVP 仿真引擎执行编译产生的仿真二进制文件，超时设为 60 秒。仿真输出的解析实现了双格式支持：对 `VerilogEval` 格式，使用正则表达式匹配 `mismatched samples is N out of M` 模式提取不匹配数和总样本数；对 RTLLM 格式，检测 `Your Design Passed`（通过）或匹配 `Test completed with N/M failures`（部分通过）。当两种格式均未匹配时，以进程返回码作为最终判断依据。仿真结果被封装为 `SimResult`



对象，包含通过标志、不匹配数、总样本数和原始输出。

3.4.4 评分机制

评分器将编译和仿真结果映射为一个 $[-2, 1]$ 区间内的标量分数（表3.2）。该评分规则借鉴了已有反馈循环研究 [5] 的设计：分数越高，代码质量越好。反馈循环控制器以 $\text{rank}=1.0$ 作为“通过”的判定阈值。

表 3.2 候选代码评分规则

情况	分数	含义
未提取到有效 Verilog	-2.0	代码提取失败
编译失败	-1.0	存在语法或结构错误
编译有警告但无仿真结果	-0.5	可能存在问题
编译成功但无仿真数据	0.0	无法评估正确性
仿真部分通过	0 ~ 1.0	(总样本 - 不匹配)/总样本
仿真完全通过	1.0	设计功能正确

3.5 反馈循环控制机制

反馈循环控制器（loop_runner.py）是系统的核心调度模块，实现了“生成—验证—反馈—修复”迭代闭环 [5]。其执行流程如算法1所示。

3.5.1 全局最优追踪

全局最优追踪机制维护一个跨迭代的历史最优候选。每当某个候选的评分超过当前全局最优时，控制器更新全局最优的代码、编译结果和仿真结果。下一轮迭代的反馈提示词始终基于全局最优候选的错误信息构建。这一设计确保了反馈循环的单调性：即使某一轮的候选整体质量暂时回退，全局最优也不会丢失。

3.5.2 Zero-shot 与 Retry-only 作为退化模式

零样本模式是反馈循环在 $k = 1$ 、 $\text{max_iterations}=1$ 时的退化形式。Retry-only 模式（ $k = 3$ 、 $\text{max_iterations}=5$ 但 $\text{no_feedback}=\text{True}$ ）则在所有迭代中均使用初始提示词，不注入任何错误反馈信息。这两种模式与完整反馈模式共享同一代码路径（run_



Algorithm 1 单轮反馈循环执行流程

Require: 任务 T (description, module_header, testbench_path)

Require: 参数 k (每轮候选数)、 N (最大迭代轮数)、 m (反馈模式)

Ensure: 最优 Verilog 代码及其评分

```
1:  $global\_best \leftarrow null, best\_rank \leftarrow -2.0$ 
2:  $prompt_0 \leftarrow BuildInitialPrompt(T)$ 
3: for  $i = 1$  to  $N$  do
4:   if  $i = 1$  or  $m = \text{retry-only}$  then
5:      $prompt \leftarrow prompt_0$ 
6:   else
7:      $prompt \leftarrow BuildFeedbackPrompt(T, global\_best, m)$ 
8:   end if
9:   for  $j = 1$  to  $k$  do
10:     $code_j \leftarrow ExtractVerilog(LLM(prompt))$ 
11:     $rank_j \leftarrow Evaluate(code_j, T.testbench)$ 
12:    if  $rank_j > best\_rank$  then
13:      更新  $global\_best, best\_rank$ 
14:    end if
15:  end for
16:  if  $best\_rank = 1.0$  then
17:    return ( $global\_best, 1.0$ ) {设计通过}
18:  end if
19: end for
20: return ( $global\_best, best\_rank$ ) {迭代耗尽}
```

Feedback Loop Control Flow

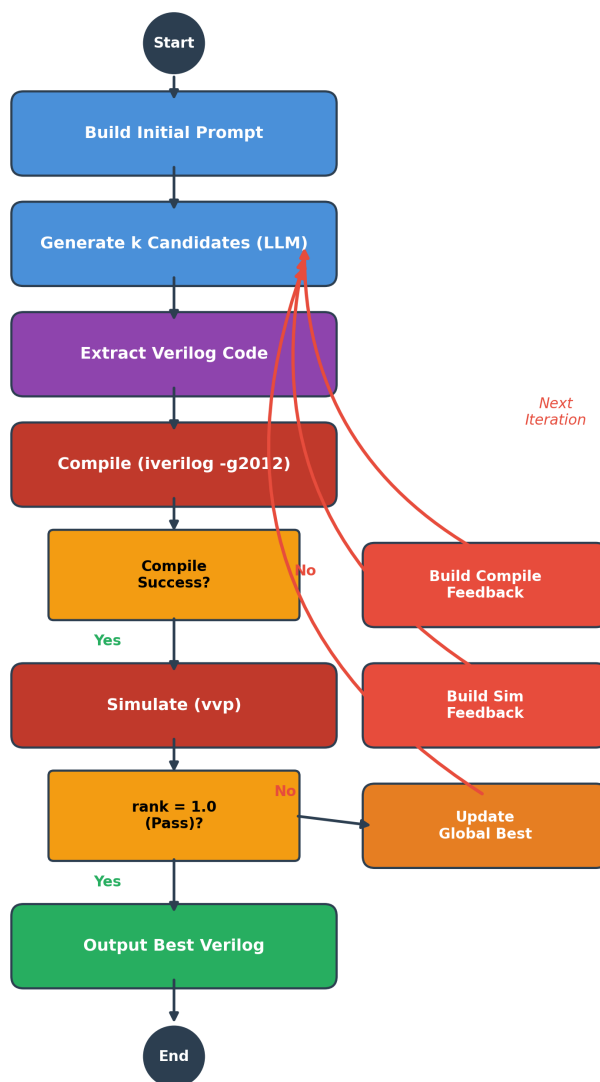


图 3.2 反馈循环控制流程



feedback_loop 函数), 确保了实验条件的一致性和对比的公平性。

3.5.3 API 错误容错

当某个候选的 API 调用失败时, 该候选被标记为 `api_error=True` 但不终止当前迭代。只有当一轮迭代中全部 k 个候选都因 API 错误而失败时, 循环才提前退出并将整个任务标记为 API 错误。这一设计在 API 稳定性不高的实验环境下尤为重要。

3.6 反馈策略扩展实现

3.6.1 五级反馈粒度

为了研究反馈信息量对修复效果的定量影响, 系统实现了五个梯度的反馈水平, 其定义如表3.3所示。

表 3.3 反馈粒度级别定义

级别	名称	模式	反馈内容
L0	Zero-shot	无反馈	仅初始提示词, 单次生成
L1	Retry-only	无反馈	每轮使用初始提示词, 多次独立重试
L2	Compile-only	编译反馈	仅包含编译阶段错误信息
L3	Succinct	简洁反馈	编译错误 + 仿真不匹配数 + 输出前 40 行
L4	Rich	丰富反馈	编译错误 + 仿真输出前 80 行 + 分析提示

其中 L0 和 L1 由反馈循环控制器通过参数控制: L0 为单次执行, L1 设置 `no_feedback=True` 禁用反馈注入。L2–L4 则由提示词构建器中的 `FeedbackMode` 枚举实现。五个级别共享相同的循环框架, 仅在反馈提示词构造方式上存在差异。

3.6.2 多轮对话反馈

除上述单轮 API 调用模式外, 系统还实现了多轮对话反馈 (Multi-turn Feedback)。在该模式下, LLM 客户端维护一个持续增长的对话历史 (messages 列表), 每次 API 调用将完整历史传入。多轮对话的关键设计约束是: 反馈消息必须描述模型在当前对话中最近一轮生成的代码的错误, 不能引用对话外的代码。本文在 v1 版本实现中曾出现反



馈信息与对话上下文不一致的 bug，导致实验数据失效；修复后的 v2 版本严格保证了反馈—上下文的一致性。

3.7 提示词策略扩展实现

3.7.1 四种提示策略

为研究初始提示词工程对 RTL 生成质量的影响，系统实现了四种可切换的提示策略 [23]：P0（Base）为直接指令式，要求模型仅返回 Verilog 代码；P1（CoT）在 P0 基础上要求模型先进行推理再输出代码；P2（Few-shot）在提示词中附加两个已解决的示例；P3（Few-shot+CoT）结合了示例和推理要求。

3.7.2 数据泄漏防护

Few-shot 示例的选取需要避免与正式实验任务重叠。系统使用的两个示例——8 位加法器（adder_8bit）和 4 位计数器（counter_4bit）——均不在 RTLLM_STUDY_12 正式研究子集中，从而避免了训练/评估数据泄漏问题。

3.7.3 策略调度与元数据记录

提示策略通过 PromptStrategy 枚举类型定义，实验脚本通过 `--prompt-strategy` 命令行参数指定。策略名称自动记录到实验元数据（metadata.json）中，确保每次运行的提示策略选择可追溯。

3.8 实验产物管理与审计机制

3.8.1 运行目录与产物文件

每次实验运行被分配一个唯一的 run_id（格式为 `< 前缀 >_< 时间戳 >`，例如 `rtllm_feedback_20260420_153022`），其产物存放在 `outputs/runs/< 类别 >/<run_id>/` 目录下。目录内包含三类产物文件：summary.json 记录运行级汇总（整体通过率、参数配置、Git 状态等）；details.json 记录每道题、每轮迭代、每个候选的完整信息；summary.csv 提供表格化的题目级结果，便于后续数据分析。



3.8.2 元数据与 Git 集成

产物管理器在每次运行开始时自动采集元数据, 包括: 当前 Git commit hash、分支名称、工作区是否有未提交修改 (dirty 标志)、Python 版本、操作系统平台、完整的命令行参数, 以及模型名称和实验参数。这些信息写入 summary.json 的 metadata 字段, 使得任何实验结果都可以精确追溯到产生它的代码版本和运行环境。

3.8.3 审计脚本

为保障实验数据的可信度, 系统提供了两个审计脚本。代码审计脚本 (audit_project.py) 执行 24 项自动化检查, 涵盖项目结构、关键文件存在性、依赖完整性和代码风格。数据审计脚本 (audit_data.py) 遍历全部实验运行目录, 从原始 summary.json 中重新计算通过率并与文档记录的结论交叉验证, 任何不一致均会触发警告。此外, 系统维护一份权威实验索引文档, 对每个实验运行标注其状态 (有效/已废弃/已替代), 防止论文撰写过程中误引用已废弃的结果。

除原始运行数据外, 各组实验的主要结果还被整理为通过率对比图、逐题结果矩阵、消融对比图、稳定性误差棒图、反馈粒度曲线和多轮反馈矩阵等图表资产, 供第 5 章结果分析直接引用。

3.9 本章小结

本章从系统架构到各模块实现细节, 完整地描述了基于 EDA 工具反馈的 RTL 自动生成与优化原型系统的设计与实现。系统通过统一的 Task 接口屏蔽了 VerilogEval 和 RTLLM 在文件格式上的差异, 通过环境变量驱动的 API 网关实现了多模型无缝切换, 通过五级反馈粒度和四种提示策略提供了灵活的实验配置能力, 并通过元数据记录和审计脚本确保了实验结果的可追溯性和可信度。围绕 8 个核心模块 (约 2100 行), 项目还配套实现了 RTLLM 兼容性扫描、多模式实验调度和结果图表整理等工作, 共同为第 4 章的实验设计和第 5 章的结果分析提供了工程基础。



4 实验设计

本章定义本文全部实验的目标、评估基准、模型配置、实验条件、评价指标与可靠性控制方法。实验设置均来自真实的实验脚本与权威文档，具体结果将在第 5 章中呈现。

4.1 实验目标与总体设计

本文的实验旨在回答以下问题：EDA 反馈循环能否在不同难度的 benchmark 和不同能力的模型上稳定提升 Verilog 代码正确率？反馈收益中多次采样和错误信息分别贡献了多少？不同粒度的反馈对修复效果有何差异？多轮对话反馈和初始提示词优化能否进一步提升表现？

为此，本文设计了三个层次共七组递进式对照实验。第一层为有效性验证：先在 VerilogEval-Human 上完成基线实验确认系统可用性，再在难度更高的 RTLLM_STUDY_12 上进行三模型正式对比。第二层为因果归因：通过 retry-only 消融条件将反馈收益分解为多采样和反馈信息两个独立来源，辅以每模型 4 轮的稳定性重复实验检验结论的统计可靠性。第三层为机制探索：从反馈信息量（粒度实验）、信息组织方式（多轮对话实验）和初始生成质量（提示词策略实验）三个维度分析优化空间。表 4.1 汇总了各研究问题与实验的对应关系。

表 4.1 研究问题与实验设计对应关系

编号	研究问题	对应实验	主要比较条件
RQ1	反馈循环是否提升正确率	基线 + 正式实验	Zero-shot vs Feedback
RQ2	反馈 vs 多次重试的贡献	消融实验	Retry-only vs Feedback
RQ3	结论是否统计稳定	稳定性重复	4 轮独立重复
RQ4	反馈粒度的影响	粒度实验	L0-L4 五级
RQ5	多轮对话是否优于单轮	多轮对话实验	ST vs MT
RQ6	提示词策略的影响	策略实验	P0-P3 四种策略
RQ7	跨模型一致性	全部实验	三个模型交叉验证



4.2 评估基准与任务子集

VerilogEval-Human 20 题子集。 VerilogEval-Human[3] 是 NVlabs 发布的 Verilog 代码生成评估基准的人工编写子集。本文在中期阶段选取其中 20 个题目作为基线实验的任务集，覆盖组合逻辑、时序逻辑、有限状态机等多种类型。

RTLLM 基准与兼容性审计。 RTLLM 2.0[4] 包含 50 个硬件设计任务。由于本文使用 Icarus Verilog[14] 作为编译仿真工具，接入前进行了全量兼容性审计，结果如表4.2所示。

表 4.2 RTLLM 2.0 Icarus Verilog 兼容性审计结果

指标	数量	比例
总设计数	50	100%
编译通过	46	92%
仿真通过	41	82%
编译失败	4	8%
编译通过但仿真异常	5	10%

RTLLM_CORE_5 打通验证子集。 在正式实验前，本文定义了一个 5 题的最小验证子集 RTLLM_CORE_5 用于端到端验证。该子集的选取标准是：涵盖不同类别（算术、控制、存储器）、编译和仿真均已确认可用、难度适中。CORE_5 的实验结果仅用于确认系统在 RTLLM 格式下的正确运行，不作为正式结论来源。

RTLLM_STUDY_12 正式研究子集。 RTLLM_STUDY_12 是全部正式实验的核心任务集，包含 12 个精心选取的题目，如表4.3所示。选取标准包括：编译和仿真均通过兼容性审计；覆盖算术运算、控制逻辑、存储器和杂项功能四大类别；难度梯度从三星到五星，确保实验能够区分不同模型和条件的差异。其中 float_multi（IEEE 754 浮点乘法器）是整个 RTLLM 基准中难度最高的设计之一，sequence_detector 和 freq_divbyfrac 则在预实验中被发现对所有模型均构成挑战。

4.3 模型与参数设置

本系统通过第三方统一 API 网关接入多种大语言模型。实验涉及三个模型，代表三个能力等级，如表4.4所示。

除另有说明外，所有实验使用以下统一参数：temperature=0.7，max_tokens=4096，每轮候选数 $k = 3$ ，最大迭代轮数为 5，默认反馈级别为 L3 Succinct，默认提示策略为



表 4.3 RTLLM_STUDY_12 任务组成与类别分布

#	设计名	类别	难度
1	float_multi	Arithmetic/Other	★★★★★
2	multi_booth_8bit	Arithmetic/Multiplier	★★★★
3	multi_pipe_8bit	Arithmetic/Multiplier	★★★★
4	div_16bit	Arithmetic/Divider	★★★★
5	adder_bcd	Arithmetic/Adder	★★★
6	fsm	Control/FSM	★★★
7	sequence_detector	Control/FSM	★★★
8	JC_counter	Control/Counter	★★★
9	LIFObuffer	Memory/LIFO	★★★
10	LFSR	Memory/Shifter	★★★
11	traffic_light	Miscellaneous/Others	★★★★
12	freq_divbyfrac	Miscellaneous/Freq divider	★★★★

表 4.4 实验模型与角色说明

模型名称	提供商	角色	实验覆盖
claude-haiku-4-5	Anthropic	较弱基线	VerilogEval 基线 + RTLLM 正式
claude-sonnet-4-6	Anthropic	中强模型	正式 + 消融 + 稳定性 + 粒度 + 多轮
gpt-5.4	OpenAI	强模型	正式 + 消融 + 稳定性 + 粒度 + 多轮 + 策略



P0 Base。

4.4 实验条件与对照设计

正式实验。正式实验矩阵在 RTLLM_STUDY_12 上对三个模型分别运行两种条件：Zero-shot ($k=1$, 不迭代, 无反馈) 和 Feedback ($k=3$, 最多 5 轮, L3 Succinct 反馈) [5]。

消融实验。为分离多次重试与反馈信息的各自贡献, 设计了三条件消融：Zero-shot (预算 1)、Retry-only (预算最多 15, 无反馈) 和 Feedback (预算最多 15, L3 反馈)。

稳定性重复实验。对消融实验进行独立重复：GPT-5.4 和 Sonnet 4.6 各 4 轮, 每轮完全独立的 API 调用, 用于评估结论的统计稳健性。

反馈粒度实验。设计五级粒度条件：L0 Zero-shot、L1 Retry-only、L2 Compile-only、L3 Succinct、L4 Rich, 用于研究反馈信息量与修复效果的关系。

多轮对话反馈实验。设计四条件对照, 在 STUDY_12 的 6 题代表性子集 (Multiturn-6) 上运行。本实验初始版本 (v1) 存在代码 bug, 已修正为 v2 版本, 本文报告的所有多轮对话结果均来自 v2。

提示词策略实验 [10, 23]。设计四种提示策略 (P0 Base、P1 CoT、P2 Few-shot、P3 Few-shot+CoT) 的对比实验, 每种策略均运行 Zero-shot 和 Feedback 两种条件, 使用 GPT-5.4 模型。

4.5 评价指标与统计方法

通过率 (Pass Rate) 是本文的首要评价指标, 定义为通过测试平台全部测试用例的任务数占总任务数的比例。Rank 分数作为辅助指标, 取值范围为 $[-2.0, 1.0]$ 。派生指标包括 Feedback 改善题数 (衡量反馈的净贡献) 和 API 错误率 (衡量实验数据完整性)。

对于稳定性重复实验, 报告 4 轮重复的均值和标准差。对于单次运行的实验, 在结论表述中注明其局限性。

4.6 实验可靠性控制

审计与验证机制包括代码审计 (24 项检查) 和数据审计 (覆盖全部 7 个实验类别), 两项均已通过。权威实验索引明确标注每个实验阶段的状态, 确保论文中每一个实验数值均可追溯。



4.7 本章小结

本章定义了全部实验方案，涵盖两个评估基准、三个模型和七组递进式实验。各实验与研究问题的对应关系参见表4.1。



5 实验结果与分析

本章依次呈现七组实验的结果并进行分析。所有数据以权威实验索引为准。

5.1 VerilogEval-Human 基线实验

本节报告中期阶段在 VerilogEval-Human 20 题子集上使用 Haiku 模型的基线实验。反馈循环将通过率从 80% 提升至 90%，具体结果如表5.1所示。在 4 道零样本未通过的题目中，反馈成功修复了 2 道（50%）。

表 5.1 VerilogEval-Human 20 题基线实验结果

条件	通过数	通过率
Zero-shot	16/20	80%
Feedback	18/20	90%
提升	+2	+10 个百分点

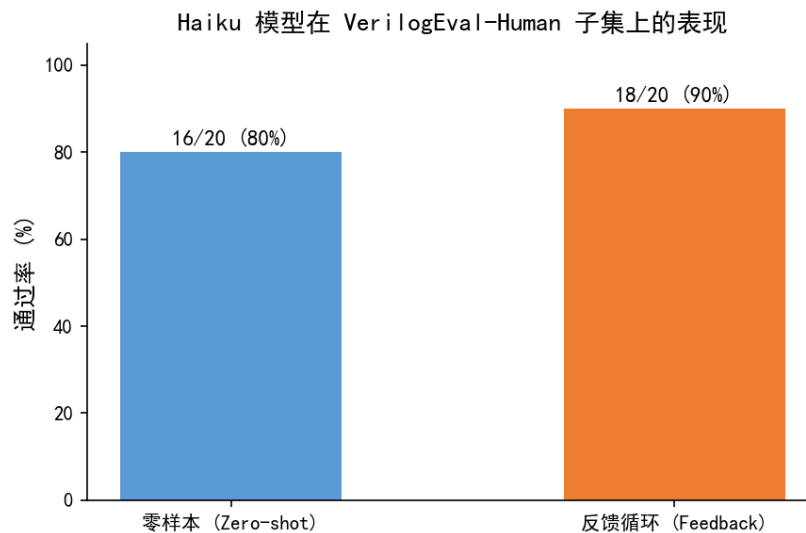


图 5.1 VerilogEval-Human 20 题零样本与反馈通过率对比

反馈成功修复的两道题为 Prob109_fsm1 和 Prob127_lemmings1，均属于代码接近正确但存在局部状态转移错误的场景——反馈循环对这类“差一步”的问题尤为有效。未能修复的两道题分别涉及 LFSR 抽头多项式（精确数学知识）和 HDLC 协议边界条件

(深层协议逻辑)，表明反馈对领域知识缺口的修复能力有限。

5.2 RTLLM_STUDY_12 正式实验结果

在 RTLLM_STUDY_12 上使用三个模型进行 Zero-shot 与 Feedback 对比，结果如表5.2所示。三组实验的 API 错误率均为零。

表 5.2 RTLLM_STUDY_12 三模型正式实验结果

模型	Zero-shot	Feedback	提升	改善题数
Haiku 4.5	5/12 (42%)	6/12 (50%)	+8pp	2
Sonnet 4.6	5/12 (42%)	7/12 (58%)	+17pp	2
GPT-5.4	6/12 (50%)	10/12 (83%)	+33pp	4

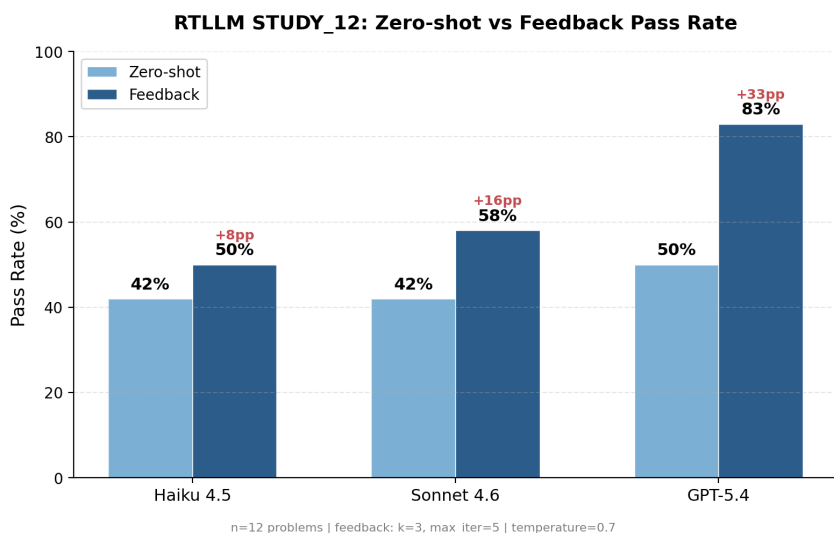


图 5.2 RTLLM_STUDY_12 三模型通过率对比

反馈对所有模型均产生了正向增益。值得注意的是，强模型从反馈中获益最大——GPT-5.4 的增益（+33 个百分点）远超 Sonnet（+17 个百分点）和 Haiku（+8 个百分点）。这一现象的可能解释是：反馈信息需要模型具备足够的代码理解和修复能力才能被有效利用。强模型在接收到编译或仿真错误信息后，更能准确定位问题根因并生成正确的修复方案；而弱模型即使获得了相同的错误信息，也可能因基础能力不足而无法将反馈转化为有效修复。换言之，反馈循环的收益上限取决于模型自身的代码推理能力。

实验还发现了两类值得关注的特殊题目。第一类是仅反馈可解的题目：LFSR 和 traffic_light 在零样本下所有模型均失败，但在反馈条件下被成功修复，说明这类题目的

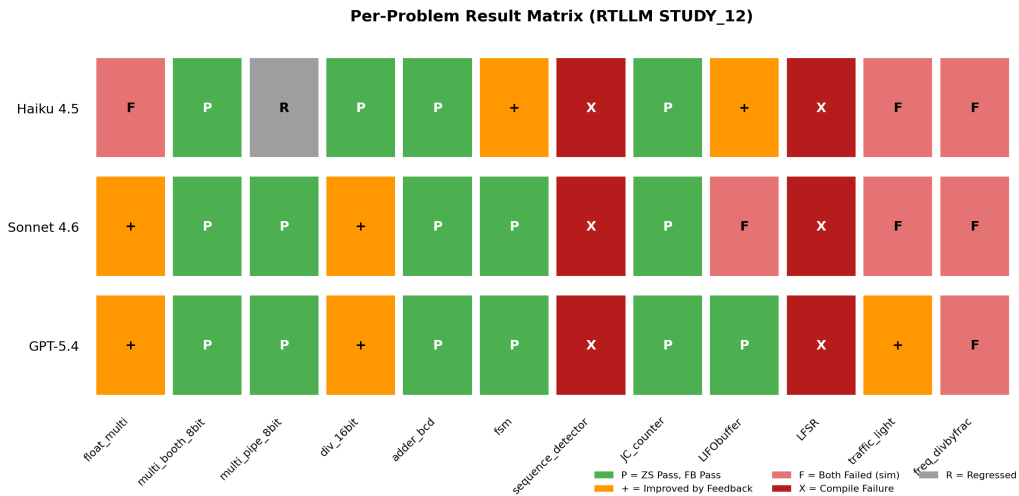


图 5.3 RTLLM_STUDY_12 逐题结果矩阵

主要障碍不是设计复杂度而是初始生成中的局部错误，反馈能够精准定位并修复这些错误。第二类是天花板题：sequence_detector 和 freq_divbyfrac 在所有模型和条件下均未通过，暴露了当前反馈框架对涉及精确算法知识的设计任务的局限。

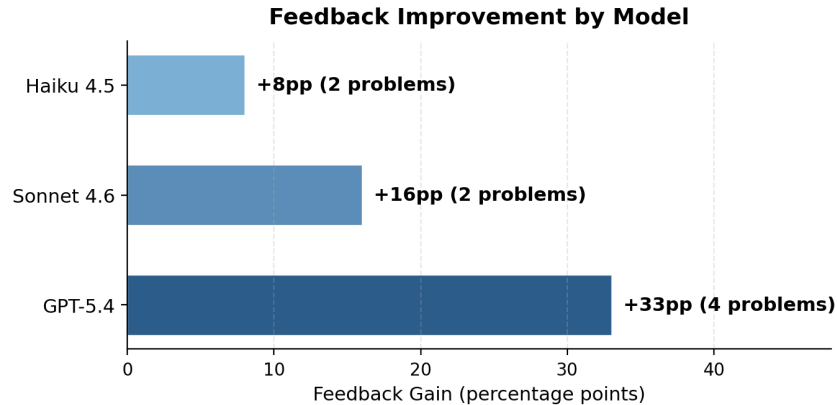


图 5.4 各模型 Feedback 增益对比

5.3 Feedback vs Retry-only 消融实验

为区分反馈循环中“多次重试”与“错误反馈信息”的各自贡献，在 GPT-5.4 和 Sonnet 4.6 上运行了三条件消融实验（实验设计详见第 4 章）。设计 retry-only 条件的核心动机在于：反馈循环同时带来了两个优势——更多的生成机会和来自 EDA 工具的错误信息，如果不加以分离，就无法判断反馈的实际价值。

GPT-5.4 消融结果。 GPT-5.4 的单轮消融结果为：Zero-shot 50% (6/12)，Retry-only 67%

(8/12), Feedback 83% (10/12)。由此可见, 从零样本到反馈的 +33 个百分点总提升中, 多采样贡献 +17 个百分点, 反馈信息贡献 +16 个百分点, 反馈信息约占总提升的 49%。需要注意的是, 单轮结果受随机波动影响, 稳定性实验 (4 轮均值, 见 5.4 节) 修正后的比例为反馈信息约占 57%。这一差异提示我们: 对于涉及随机采样的实验, 单轮数据应谨慎解读。

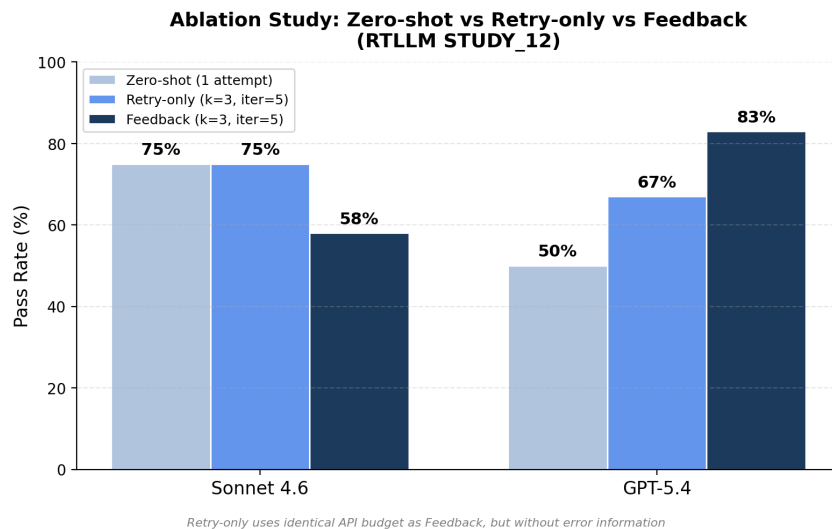


图 5.5 零样本/重试/反馈三条件对比

消融实验中最具说服力的证据来自仅反馈可解的题目: `sequence_detector` 和 `traffic_light` 在 15 次独立重试中从未通过, 但在反馈条件下于 2–3 轮内成功修复, 直接说明错误反馈信息提供了独立采样无法获得的修复信号。

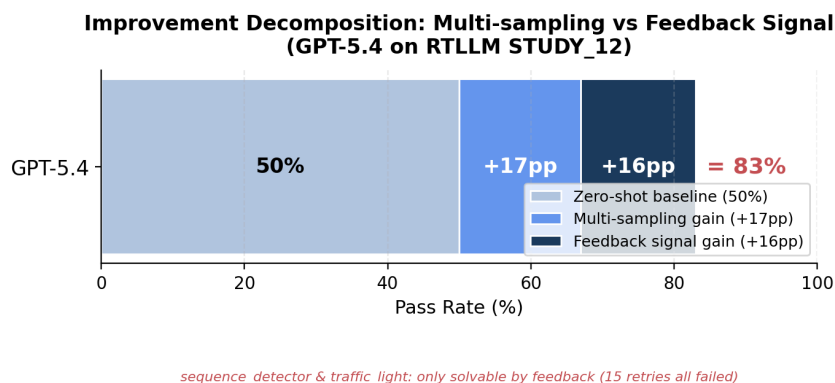


图 5.6 GPT-5.4 反馈收益来源分解

Sonnet 4.6 消融结果。 Sonnet 4.6 的单轮消融结果显示 Feedback (58%) 低于 Retry-only (75%)。这一看似反直觉的结果意味着: 对 Sonnet 而言, 在当前配置下, 与其让模型依

据反馈信息尝试修复，不如让其从头重新生成。可能的原因是 Sonnet 在处理反馈信息时倾向于对已正确部分进行过度修改，导致修复引入新的错误。该结论是否稳定，需结合下一节的稳定性实验综合判断。

5.4 稳定性重复实验

消融实验的结论是否具有统计可靠性？单轮实验受 LLM 生成随机性影响较大，因此本节通过 4 轮独立重复来检验核心结论的稳定性。每轮使用相同的参数配置但不同的随机种子，产生完全独立的实验结果。

GPT-5.4。4 轮独立重复中，Feedback 始终优于 Retry-only（4/4 轮），且无一例外。反馈不仅提升了平均通过率，还将标准差从 11.8%（零样本）压缩至 4.2%，表明反馈循环不仅提高了成功率，还降低了结果的波动性，如表 5.3 所示。

表 5.3 GPT-5.4 稳定性实验统计

统计量	Zero-shot	Retry-only	Feedback
均值	50.0%	62.5%	79.2%
标准差	$\pm 11.8\%$	$\pm 4.2\%$	$\pm 4.2\%$
FB>RO 轮次	—	—	4/4

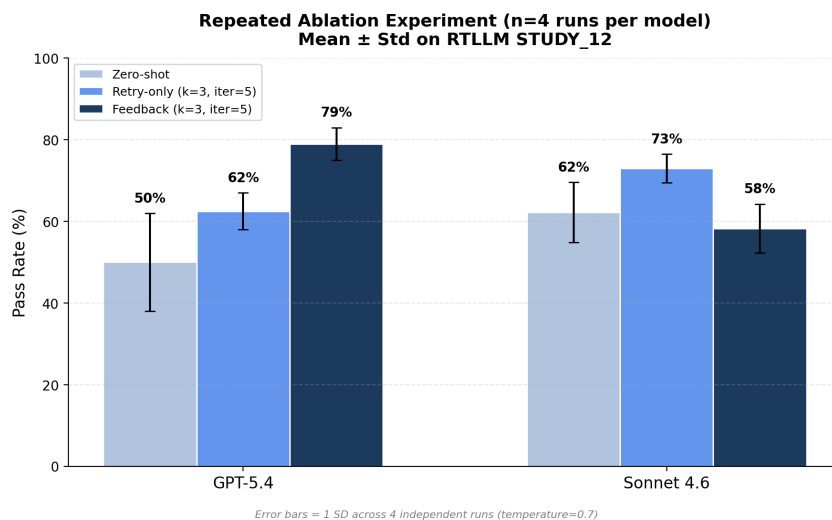


图 5.7 稳定性实验三条件均值与标准差

Sonnet 4.6。Sonnet 上 Feedback（均值 58.3%±5.9%）低于 Retry-only（72.9%±3.6%）的结论在 4 轮中均成立（0/4 轮 FB 优于 RO），如表 5.4 所示。

表 5.4 Sonnet 4.6 稳定性实验统计

统计量	Zero-shot	Retry-only	Feedback
均值	62.5%	72.9%	58.3%
标准差	$\pm 7.2\%$	$\pm 3.6\%$	$\pm 5.9\%$
FB>RO 轮次	—	—	0/4

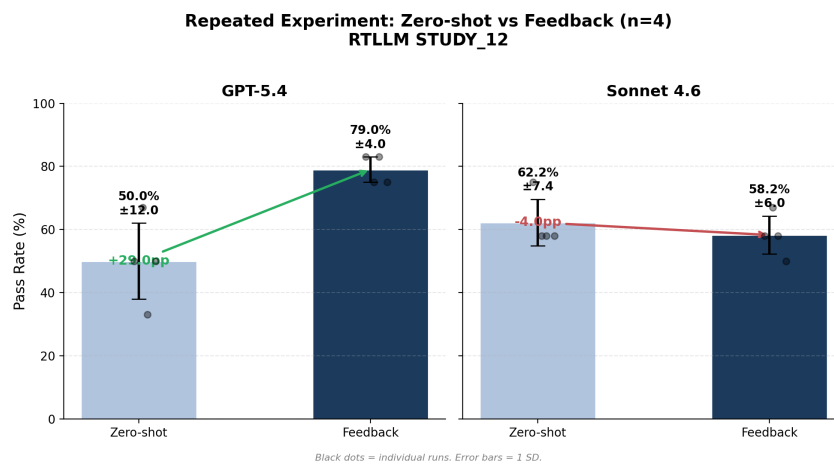


图 5.8 正式实验重复结果对比

模型依赖性。两个模型对反馈的响应截然相反：GPT-5.4 上 FB 相对 RO 的增益为 +16.7 个百分点，Sonnet 上为 -14.6 个百分点。这一分化在 4 轮重复中完全稳定（标准差均小于 6%），说明这不是随机噪声而是两个模型在利用反馈信息方面的本质差异。这一发现对实际部署具有直接指导意义：在选择是否启用反馈循环之前，应先在目标模型上进行小规模验证。

5.5 反馈粒度实验

反馈中应包含多少信息？直觉上信息越多越好，但过多的信息可能干扰模型对关键错误的定位。本节通过五级粒度实验（L0-L4）系统测试信息量对修复效果的影响。结果如表5.5所示，呈现出倒 U 型趋势：通过率随反馈信息量的增加先升后降，L2（Compile-only）和 L3（Succinct）达到峰值，而 L4（Rich）反而出现下降。

从实践角度看，L1→L2 是最大的跃升点（GPT-5.4 上从 75% 跃升至 92%），表明即使是最基础的编译错误反馈也具有极高的修复价值。L2 Compile-only 在两个模型上均为最优或并列最优，是最稳健的通用选择。L4 Rich 反馈包含多达 80 行仿真输出和分析提示，其通过率反而低于 L2 和 L3，可能的原因是：过于冗长的仿真日志稀释了关键错

表 5.5 反馈粒度实验通过率

级别	GPT-5.4	Sonnet 4.6
L0 Zero-shot	58% (7/12)	50% (6/12)
L1 Retry-only	75% (9/12)	67% (8/12)
L2 Compile-only	92% (11/12)	75% (9/12)
L3 Succinct	92% (11/12)	67% (8/12)
L4 Rich	83% (10/12)	67% (8/12)

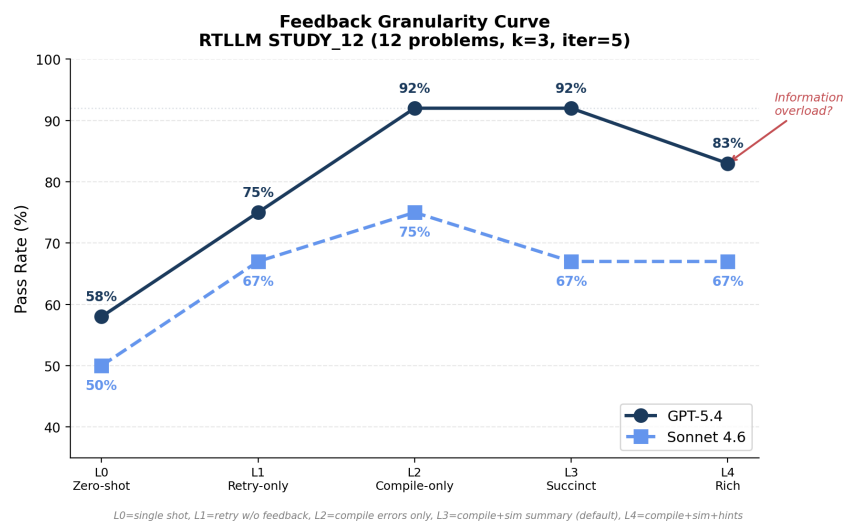


图 5.9 反馈粒度通过率曲线（倒 U 型）

误信息的信噪比，使模型难以从中提取修复所需的核心线索。这一“信息过载”现象与信息论中的有限注意力模型一致。

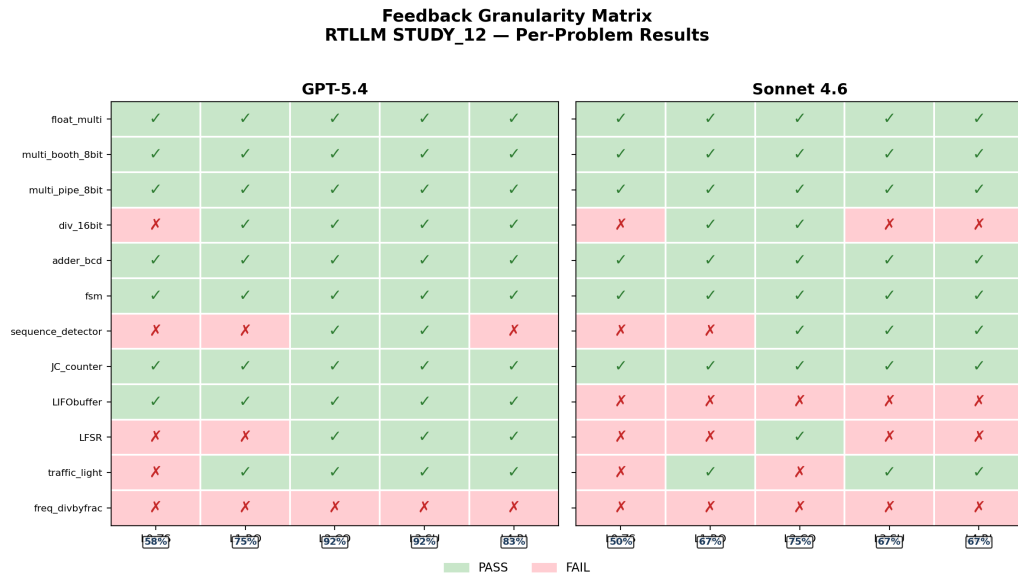


图 5.10 反馈粒度逐题结果矩阵

5.6 多轮对话反馈实验

前述实验均采用单轮 API 调用模式，即每次反馈都发起一次独立请求。本节探索将反馈组织为多轮对话（Multi-turn），使模型能够看到完整的修复历史。本节结果均来自 v2 修正版。

GPT-5.4 和 Sonnet 4.6 的多轮对话实验结果分别如表5.6和表5.7所示。在控制候选数量的公平对比下（ST $k=1$ vs MT $k=1$ ），GPT-5.4 从 83% 提升至 100%（+17 个百分点），Sonnet 4.6 从 33% 提升至 50%（+17 个百分点），两个模型的提升幅度一致。这一结果表明，持续的对话上下文有助于模型追踪修复历史和错误演变模式，避免在不同轮次中重复相同的错误路径。需要注意的是，MT $k=1$ 与 ST $k=3$ 的比较并不公平（候选数量不同），仅可作为参考。此外，该实验在 6 题子集上进行且为单次运行，统计效力有限，结论需谨慎推广。

5.7 提示词策略实验

初始提示词的质量是否影响最终代码正确率？本节在 GPT-5.4 上对比四种提示策略（Base、CoT、Few-shot、Few-shot+CoT），分析提示词工程与反馈循环的交互关系。结果



表 5.6 多轮对话反馈 v2 实验结果 (GPT-5.4)

条件	通过率
A: ST $k=3$	83% (5/6)
D: ST $k=1$	83% (5/6)
B: MT $k=1$	100% (5/5)*
C: CO $k=3$	83% (5/6)

*freq_divbyfrac 因 API 超时排除。

表 5.7 多轮对话反馈 v2 实验结果 (Sonnet 4.6)

条件	通过率
A: ST $k=3$	17% (1/6)
D: ST $k=1$	33% (2/6)
B: MT $k=1$	50% (3/6)
C: CO $k=3$	33% (2/6)

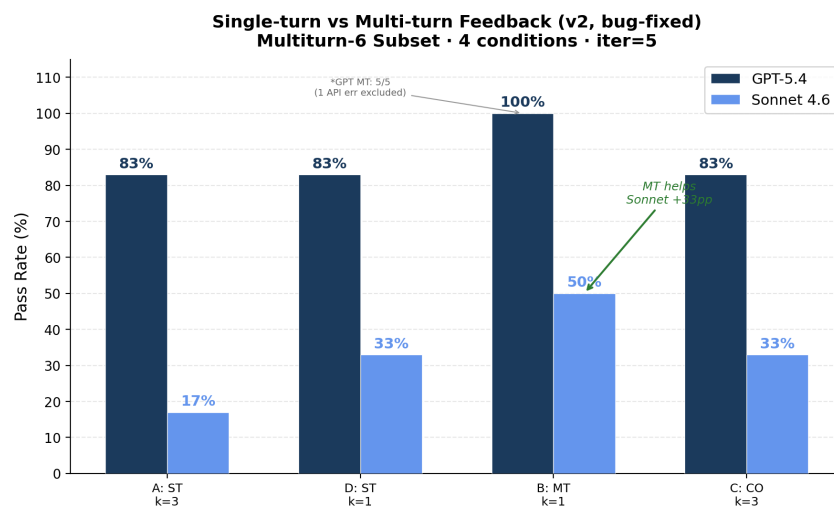


图 5.11 多轮对话反馈 v2 条件对比

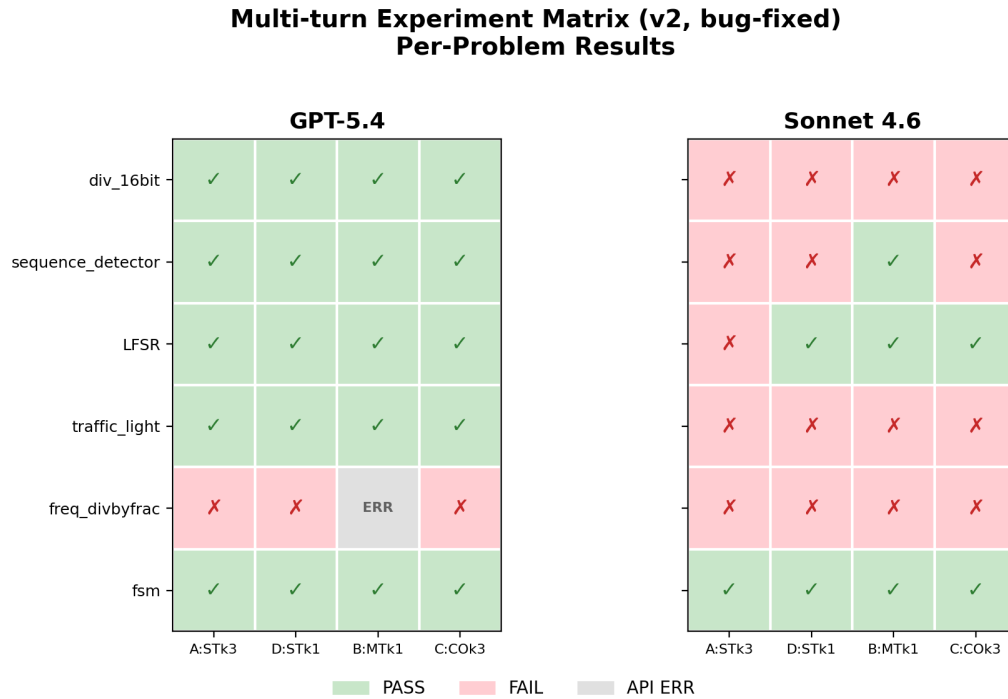


图 5.12 多轮对话反馈 v2 逐题矩阵

如表5.8所示。

表 5.8 提示词策略实验结果（GPT-5.4）

策略	Zero-shot	Feedback	FB 增益
P0: Base	58% (7/12)	92% (11/12)	+34pp
P1: CoT	50% (6/12)	92% (11/12)	+42pp
P2: Few-shot	67% (8/12)	92% (11/12)	+25pp
P3: Few-shot+CoT	58% (7/12)	92% (11/12)	+34pp

在零样本阶段，Few-shot（P2）以 67% 成为最优策略，说明提供具体示例有助于模型理解 Verilog 代码结构。CoT（P1）反而略低于基线，可能因为链式推理在 RTL 生成任务中增加了不必要的中间步骤，引入了额外的出错机会。

然而引入反馈后，四种策略全部收敛至 92%（11/12），仅 freq_divbyfrac 在所有策略下均未通过。这一“拉平效应”说明，反馈循环是决定最终代码质量的主导因素，初始提示词策略更多影响零样本起点而非经过反馈修复后的最终上限。从实践角度看，这意味着在配备了反馈循环的系统中，精细的提示词工程的边际收益有限，系统设计者可以将更多精力投入到反馈信息的构建上。

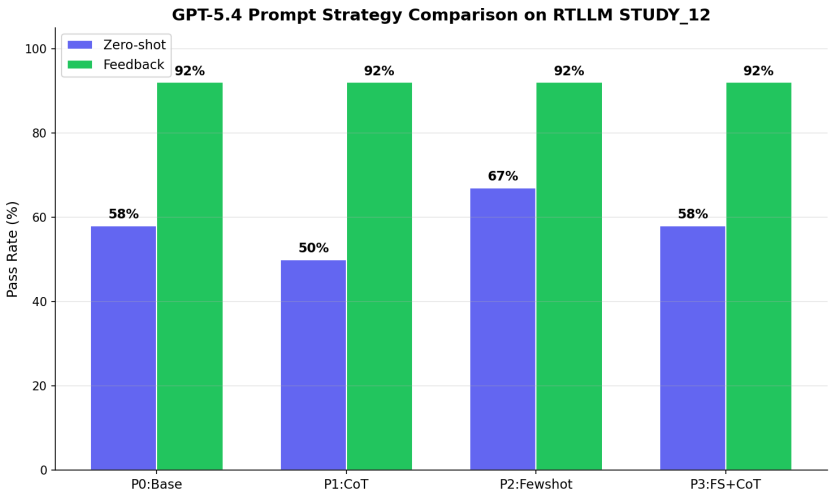


图 5.13 提示词策略零样本与反馈对比

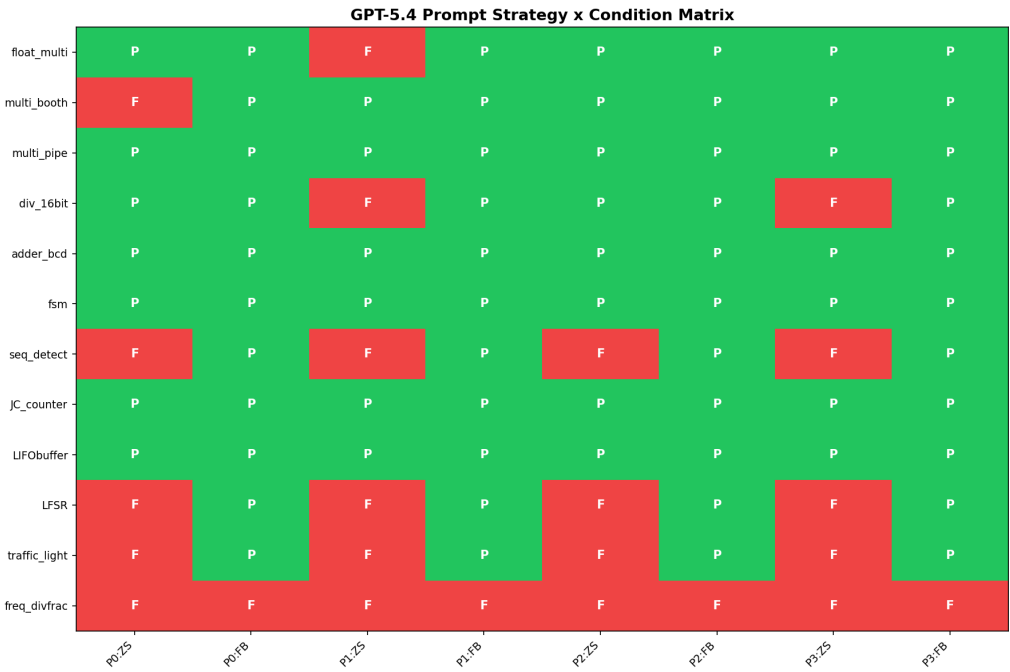


图 5.14 提示词策略逐题结果矩阵



5.8 典型案例分析

float_multi: 反馈修复最高难度设计。 float_multi (IEEE 754 浮点乘法器) 是 STUDY_12 中难度最高的题目, 要求实现符号位处理、指数加法、尾数乘法和规格化等多个步骤。三个模型在零样本下全部失败, 典型的错误模式包括指数偏移计算错误和尾数乘积截断方式不正确。然而, Sonnet 和 GPT-5.4 均在反馈的第 1 轮迭代中成功修复——编译错误信息准确指出了语法问题, 仿真反馈则帮助模型定位了具体的数值计算偏差。Haiku 即使有反馈仍无法修复, 表明该题的修复需要模型具备一定的浮点运算先验知识, 反馈信息本身不足以弥补基础能力的差距。

LIFObuffer: 反馈弥补模型差距。 LIFObuffer (后进先出缓冲区) 展示了反馈机制的另一种价值。GPT-5.4 零样本即通过, 说明该题对强模型不构成挑战。但 Haiku 在零样本下生成的代码存在读写指针管理错误, 反馈循环在第 1 轮提供了仿真输出中的指针越界信息, Haiku 据此成功修复, 追平了 GPT-5.4 的零样本水平。这一案例说明反馈机制在资源受限场景下提供了“以计算换准确度”的可行路径——使用较弱但成本更低的模型配合反馈循环, 可以达到与强模型零样本相当的效果。

adder_bcd 与 div_16bit: 反馈的稳定收益。 adder_bcd (BCD 加法器) 和 div_16bit (16 位除法器) 是两道中等难度的算术题目。在正式实验中, 这两道题在多个模型上呈现出一致的规律: 零样本下编译通过但仿真部分失败, 反馈 1-2 轮后成功修复。仿真反馈中的不匹配样本数为模型提供了明确的修复方向, 这类“功能接近正确但存在边界错误”的场景是反馈循环最典型的应用场景。

天花板题分析。 sequence_detector 在正式实验中所有模型和条件均编译失败, freq_divbyfrac 在所有条件下均未通过仿真。对 sequence_detector 的错误日志分析表明, 模型生成的代码在模块接口上与测试平台不匹配, 且多轮反馈未能纠正这一结构性问题。然而, sequence_detector 在后续的消融稳定性实验和提示词策略实验中被成功修复, 说明该题并非绝对不可解, 而是对初始生成路径高度敏感——不同的随机种子或提示词可能导致截然不同的初始代码结构。freq_divbyfrac (分数分频器) 则涉及精确的小数分频算法, 属于当前框架难以仅通过反馈解决的领域知识缺口问题。

5.9 综合讨论

综合七组实验的结果, 可以从以下几个维度进行讨论。

在有效性方面, 反馈循环在 VerilogEval-Human 和 RTLLM_STUDY_12 两个 bench-



mark 上均表现出正向增益。GPT-5.4 在 RTLLM 上的增益达到 +33 个百分点，且在 4 轮重复中稳定存在 ($79.2\%\pm4.2\%$)。从实际应用角度看，这意味着在高难度设计任务上，引入反馈循环可以将强模型的通过率从约 50% 提升到约 80%，显著降低了人工干预的需求。

在收益来源方面，消融和稳定性实验揭示了反馈收益的双重来源：约 43% 来自多次采样机会，约 57% 来自错误反馈信息本身。后者的独立价值通过仅反馈可解的题目得到了直接验证——这些题目在 15 次独立重试中从未通过，但在 2–3 轮反馈后成功修复，说明反馈信息提供了采样无法替代的修复信号。

在模型依赖性方面，GPT-5.4 上反馈带来稳定的 +16.7 个百分点增益，而 Sonnet 4.6 上反馈反而导致 -14.6 个百分点的回退，两个结论均在 4 轮重复中完全稳定。这一发现具有重要的实践意义：在部署反馈循环系统时，不能假设反馈对所有模型都有效，需要针对具体模型进行验证。Sonnet 上反馈无效的可能原因包括：该模型可能对反馈信息中的噪声更敏感，或者其修复策略倾向于过度修改已正确的部分。

在信息量边界方面，粒度实验揭示了倒 U 型趋势，编译反馈 (L2) 是最稳健的通用选择。过于丰富的仿真输出可能干扰模型对关键错误的定位，反而不如简洁的编译错误信息有效。多轮对话和提示词策略可作为补充优化方向，其中反馈循环将所有提示词策略拉平至 92%，说明反馈是最终代码质量的主导因素。

与已有工作的关系。本文的反馈循环机制受到 AutoChip[5] 等已有工作的启发。与 AutoChip 侧重于验证反馈循环的可行性不同，本文更关注该机制在更难基准、不同能力模型和不同反馈策略下的系统性表现。表5.9从六个维度总结了本文工作相对已有 feedback-in-the-loop 思路的扩展。

表 5.9 本文系统与已有反馈循环工作的对比

维度	已有工作侧重	本文工作
任务基准	可行性验证	VerilogEval + RTLLM_STUDY_12
模型覆盖	单/少量模型	Haiku / Sonnet 4.6 / GPT-5.4
反馈收益归因	关注总体提升	Zero-shot / Retry-only / Feedback 消融
反馈策略	单一反馈模式	L0–L4 五级粒度 + 多轮对话
结论可靠性	较少重复验证	4 轮独立重复实验
实验管理	结果记录	元数据 + 权威索引 + 审计脚本



因此，本文并非停留在对已有闭环思路的复现，而是在实验规模、对照条件和结果可追溯性方面做出了扩展。

表5.10汇总了各实验的主要结论。

表 5.10 主要实验结论汇总

实验	关键对比	主要结论
基线实验	Zero-shot vs FB	通过率 80%→90%
正式实验	三模型对比	GPT-5.4 增益最大 (+33pp)
消融实验	FB vs Retry-only	反馈信息贡献 ~57%
稳定性	4 轮重复	GPT-5.4 结论稳定 (4/4)
粒度实验	L0-L4	倒 U 型，L2 最稳健
多轮对话	ST vs MT	MT 有优势，需扩大验证
策略实验	P0-P3	反馈拉平至 92%

局限性。 RTLLM_STUDY_12 为 12 题子集，虽然覆盖了四大类别和多个难度梯度，但不能完全代表所有类型的 RTL 设计任务。部分实验（多轮对话、提示词策略）为单次运行或在小子集上进行，统计效力有限。Sonnet 上反馈无效的深层原因尚未完全明确，可能需要更细粒度的错误分析来回答。天花板题暴露了当前框架对涉及精确算法知识的设计任务的局限。此外，实验仅使用 Icarus Verilog 作为仿真工具，商业 EDA 工具可能提供更丰富的诊断信息。

5.10 本章小结

本章通过七组递进式实验验证了 EDA 工具反馈循环在 LLM Verilog 生成中的有效性和边界条件。核心发现包括：反馈循环在两个 benchmark 上均有效且 GPT-5.4 上增益最大；反馈信息贡献约占总提升的 57%；反馈效果具有模型依赖性；反馈信息量存在倒 U 型边界；多轮对话和提示词策略为补充优化方向。本文在实验规模、消融归因、稳定性验证和反馈粒度探索上对已有反馈循环工作进行了系统性扩展。



6 总结与展望

6.1 本文工作总结

本文围绕“基于 EDA 工具反馈的 LLM Verilog 生成与自动优化”这一问题，完成了从系统实现到实验分析的完整研究闭环。

在系统方面，构建了一个基于 EDA 工具反馈的 LLM Verilog 生成与自动优化原型系统，实现了完整的“生成—编译—仿真—反馈—修复”闭环。系统支持 VerilogEval 和 RTLLM 双 benchmark 适配、多模型统一调用、可配置的反馈粒度和提示词策略，以及完善的实验产物管理机制。围绕该系统，项目还完成了 RTLLM 兼容性扫描、多模式实验运行脚本和结果图表整理等配套工作，为多组对照实验提供了工程支撑。

在基准接入方面，对 RTLLM 2.0 全部 50 个设计进行了 Icarus Verilog 兼容性审计 (41/50 完全可用)，从中精选 12 道高难度、多类别覆盖的设计构成 RTLLM_STUDY_12 正式研究子集。

在实验方面，设计并执行了七组递进式实验，涵盖基线验证、正式对比、消融分析、稳定性验证、反馈粒度、多轮对话和提示词策略等维度。同时建立了包含元数据记录、数据审计和权威实验索引的实验管理体系。

6.2 主要实验结论

基于七组实验的结果，本文的核心发现可以概括为以下几点。

EDA 工具反馈能够提升 RTL 代码生成正确率。在 VerilogEval-Human 上，Haiku 模型的通过率从 80% 提升至 90%；在 RTLLM_STUDY_12 上，GPT-5.4 从 50% 提升至 83%，绝对提升达 33 个百分点。反馈的收益并非简单的多次重试：消融实验表明反馈信息本身贡献了约 57% 的提升，且存在仅反馈可解而 15 次独立重试均无法通过的题目。

反馈效果具有模型依赖性，GPT-5.4 上反馈始终优于 Retry-only (4/4 轮)，而 Sonnet 4.6 上反馈在当前设置下反而不如 Retry-only (0/4 轮)。反馈信息量存在最佳区间，粒度实验呈现倒 U 型趋势，编译反馈和简洁反馈是最稳健的选择。多轮对话反馈和提示词策略可作为补充优化方向。



6.3 系统局限性

本文的实验和系统存在以下局限。RTLLM_STUDY_12 包含 12 道精选题目，虽然覆盖四大类别和较高难度梯度，但不能代表所有类型的 RTL 设计任务。正式实验集中在三个模型上，其他模型的反馈响应模式尚未验证。当前系统的反馈信息主要基于仿真文本输出，缺少更精确的波形级定位。部分实验为局部探索，样本量较小，统计效力有限。

6.4 后续工作展望

当前工作留下了若干值得深入研究的方向。

在基准覆盖方面，可将实验范围从 RTLLM_STUDY_12 扩展到更大的任务子集乃至全量 RTLLM，并引入更多高难度 RTL 生成基准以验证结论的普适性。当前 12 题子集虽然覆盖了四大类别，但对某些特定领域（如通信协议、DSP 处理链）的覆盖不足，扩大任务范围有助于发现反馈机制在不同设计领域的差异化表现。

在模型覆盖方面，可纳入更多闭源和开源大语言模型进行对照。当前实验发现的模型依赖性现象（GPT-5.4 上反馈有效而 Sonnet 上无效）提示不同模型对反馈信息的处理机制可能存在本质差异。通过扩大模型覆盖范围，有望总结出哪些模型特性有利于反馈利用，并据此探索根据模型特性和错误类型动态选择反馈策略的自适应机制。

在反馈精度方面，当前系统的仿真反馈主要基于文本级的仿真输出摘要，未能利用波形级的信号差异信息。后续可引入信号级差异分析，将仿真反馈从“第 N 个样本不匹配”提升到“信号 X 在时刻 T 的期望值与实际值不一致”的精度，这种更精确的反馈有望进一步提升修复效率。

在知识增强方面，天花板题（如 `freq_divbyfrac`）暴露了当前框架的一个根本局限：当模型缺乏特定算法的先验知识时，仅靠反馈信息难以引导模型发现正确的实现方案。针对这一问题，可探索通过检索增强 [7]——在修复过程中检索相似的 Verilog 设计实现作为参考——来补充模型的领域知识。

在验证方法方面，可探索将形式化验证工具集成到反馈循环中。形式化工具能够提供反例（counterexample），即导致设计行为不符合规格的具体输入序列，这些反例可作为比仿真输出更精确的修复线索。



6.5 本章小结

本章总结了本文在系统实现和实验分析方面完成的工作，概括了主要结论，讨论了系统和实验的局限性，并从基准扩展、模型覆盖、反馈精细化、知识增强和形式化验证等方面展望了后续研究。



结论

本文构建了一个基于 EDA 工具反馈的 LLM Verilog 生成与自动优化原型系统，并通过七组递进式实验系统验证了反馈循环在不同模型和不同难度基准上的有效性与边界条件。

实验表明，反馈循环在 VerilogEval-Human 和 RTLLM_STUDY_12 两个 benchmark 上均能提升大语言模型生成 Verilog 代码的正确率。其中 GPT-5.4 在 RTLLM_STUDY_12 上的通过率从 50%（零样本）提升至 83%（反馈），增益最大且在 4 轮重复实验中稳定（ $79.2\% \pm 4.2\%$ ）。消融实验将反馈收益分解为多采样贡献（约 43%）和反馈信息贡献（约 57%），且存在仅反馈可解而多次独立重试均无法通过的设计题目。

实验同时揭示了反馈机制的边界：反馈效果具有模型依赖性；反馈信息量存在倒 U 型边界，编译反馈是最稳健的选择；天花板题暴露了当前框架对复杂算法生成的局限。多轮对话反馈和提示词策略可作为补充优化方向。

本文的工作为理解 EDA 工具反馈在大语言模型 RTL 自动生成中的作用提供了实证分析和工程参考。



致谢

毕业设计即将画上句号，回想从选题到实验再到论文定稿的这大半年，心里感慨不少。

最想感谢的是我的指导教师王锐老师。选题阶段王老师帮我理清了“大语言模型 + 硬件设计”这个方向的切入点，中期答辩时一针见血地指出应当引入更强的模型和更难基准来验证结论——正是这条建议让我后来转向 RTLLM 基准并设计了多模型交叉实验，整篇论文的实验框架也由此成型。论文修改阶段王老师在立论、结构和表述上给出了许多具体而中肯的意见，每次讨论都让我受益很大。在此向王老师致以最诚挚的谢意。

感谢北京航空航天大学计算机学院四年来提供的学习平台和资源，让我在本科阶段就有机会接触前沿研究并完成一份相对完整的实验性工作。

感谢一起度过大学四年的同学和朋友们。课上课下的交流讨论、实验室里互相帮忙调试的日子，是这段时光里最珍贵的部分。

最后，感谢我的家人。是你们一直以来的理解、包容和支持，让我能够安心投入学业。无论未来走向哪里，家永远是最坚实的后盾。



参考文献

- [1] CHANG K, WANG Y, REN H, et al. ChipGPT: How far are we from natural language hardware design[J]. arXiv preprint arXiv:2305.14019, 2023.
- [2] THAKUR S, AHMAD B, et al. Verigen: A large language model for Verilog code generation[C]//Proceedings of the Design Automation and Test in Europe Conference (DATE). Antwerp, Belgium: IEEE, 2023.
- [3] LIU M, PINCKNEY N, KHAILANY B, et al. VerilogEval: Evaluating large language models for Verilog code generation[C]//Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD). San Francisco, CA: IEEE, 2023.
- [4] LU Y, LIU S, ZHANG Q, et al. RTLLM: An open-source benchmark for design RTL generation with large language model[C]//Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC). Incheon, Korea: IEEE, 2024.
- [5] THAKUR S, AHMAD B, FAN Z, et al. AutoChip: Automating HDL generation using LLM feedback[C]//Proceedings of the ML for Systems Workshop at NeurIPS. New Orleans, LA: [s.n.], 2023.
- [6] TSAI Y D, LIU M, REN H. RTLFixer: Automatically fixing RTL syntax errors with large language models[J]. arXiv preprint arXiv:2311.16543, 2024.
- [7] YAO X, et al. HDLdebugger: Streamlining HDL debugging with large language models [J]. arXiv preprint arXiv:2403.11671, 2025.
- [8] IEEE. IEEE standard for Verilog hardware description language: IEEE Std 1364-2005[S]. New York, NY: IEEE, 2006.
- [9] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2017.
- [10] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2020.
- [11] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code



- [J]. arXiv preprint arXiv:2107.03374, 2021.
- [12] ROZIÈRE B, GEHRING J, GLOECKLE F, et al. Code llama: Open foundation models for code[J]. arXiv preprint arXiv:2308.12950, 2023.
- [13] OUYANG L, WU J, JIANG X, et al. Training language models to follow instructions with human feedback[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2022.
- [14] WILLIAMS S. Icarus Verilog[EB/OL]. 2024. <https://github.com/steveicarus/iverilog>.
- [15] WOLF C, GLASER J. Yosys — a free Verilog synthesis suite[C]//Proceedings of the Austrochip Workshop. Linz, Austria: [s.n.], 2013.
- [16] HDLBits. HDLBits — Verilog practice[EB/OL]. 2024. <https://hdlbits.01xz.net/>.
- [17] PINCKNEY N, LIU M, KHAILANY B, et al. Revisiting VerilogEval: Newer LLMs, in-context learning, and specification-to-RTL tasks[J]. arXiv preprint arXiv:2408.11053, 2024.
- [18] LIU S, FANG W, LU Y, et al. RTLcoder: Fully open-source and efficient LLM-assisted RTL code generation technique[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2024.
- [19] MENZA M U I, HASSOUN S. AIvrl: Ai-driven RTL generation with verification-in-the-loop[J]. arXiv preprint arXiv:2409.11411, 2024.
- [20] HO C T, KUO C L, KHAILANY B, et al. VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST) tools[J]. arXiv preprint arXiv:2408.08927, 2024.
- [21] BLOCKLOVE J, GARG S, KARRI R, et al. Chip-chat: Challenges and opportunities in conversational hardware design[C]//Proceedings of the ML for Systems Workshop at NeurIPS. New Orleans, LA: [s.n.], 2023.
- [22] FAN R, CHEN Y, ZHAO J, et al. AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design[C]//Proceedings of the ML for Systems Workshop at NeurIPS. Vancouver, Canada: [s.n.], 2024.
- [23] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2022.



附录 A 实验配置与运行命令

本附录记录了本文实验使用的典型运行命令模板和关键配置。

A.1 正式实验运行命令

Zero-shot

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode zero-shot
```

Feedback (default: k=3, iter=5, succinct)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode feedback
```

Ablation (zero-shot + retry-only + feedback)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode ablation
```

Granularity (L0-L4)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode granularity
```

Multiturn (ST/MT/CO conditions)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode multiturn
```



```
# Prompt strategy (P0-P3, ZS+FB each)
python scripts/run_rtlml_subset.py \
    --model gpt-5.4 --subset study12 \
    --mode prompt_strategy
```

A.2 环境配置

实验环境：Python 3.11，Icarus Verilog 12.0，Ubuntu 22.04。API 通过第三方统一网关接入。

A.3 核心代码模块索引

表 A.1 系统核心代码模块索引

模块	文件路径
反馈循环控制器	src/feedback/loop_runner.py
提示词构建器	src/feedback/prompt_builder.py
Verilog 执行器	src/runner/verilog_executor.py
评分器	src/ranking/ranker.py
RTLLM 加载器	src/runner/rtllm_loader.py
VerilogEval 加载器	src/runner/verilogeval_loader.py
LLM 客户端	src/llm/client.py
Verilog 提取器	src/utis/extract_verilog.py
实验产物管理	src/utis/artifacts.py
正式实验运行脚本	scripts/run_rtlml_subset.py
RTLLM 兼容性扫描	scripts/scan_rtlml_iverilog.py
代码审计	scripts/audit_project.py
数据审计	scripts/audit_data.py